

CSCE 611

Advanced Digital Design



Logic Circuits and SystemVerilog



South Carolina

CSCE 211 Review

Behavioral

ex2] $X = (\underbrace{A\bar{B}(C+BD)}_{\text{dist}} + \bar{A}\bar{B})C$

$\downarrow \text{dist}$

$(\bar{A}\bar{B}C + \bar{A}\bar{B}BD + \bar{A}\bar{B})C$

$\downarrow \text{dist}$

$\bar{A}\bar{B}CC + \bar{A}\bar{B}BDC + \bar{A}\bar{B}C$

$\downarrow \#3$

$\bar{A}\bar{B}C + \bar{A}\bar{B}BDC + \bar{A}\bar{B}C$

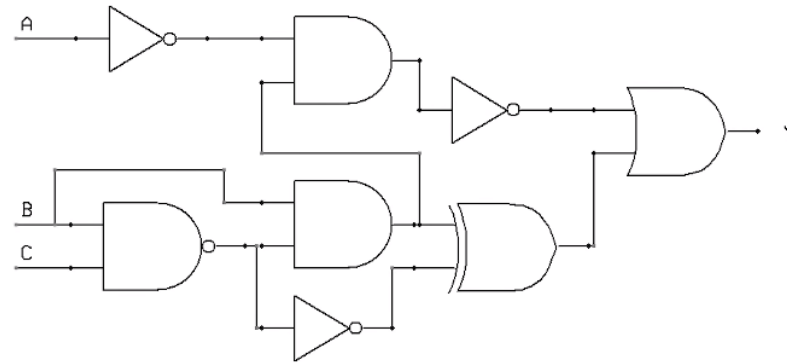
$\downarrow \#1$

$\bar{A}\bar{B}C + \bar{A} \cdot 0 \cdot DC + \bar{A}\bar{B}C$

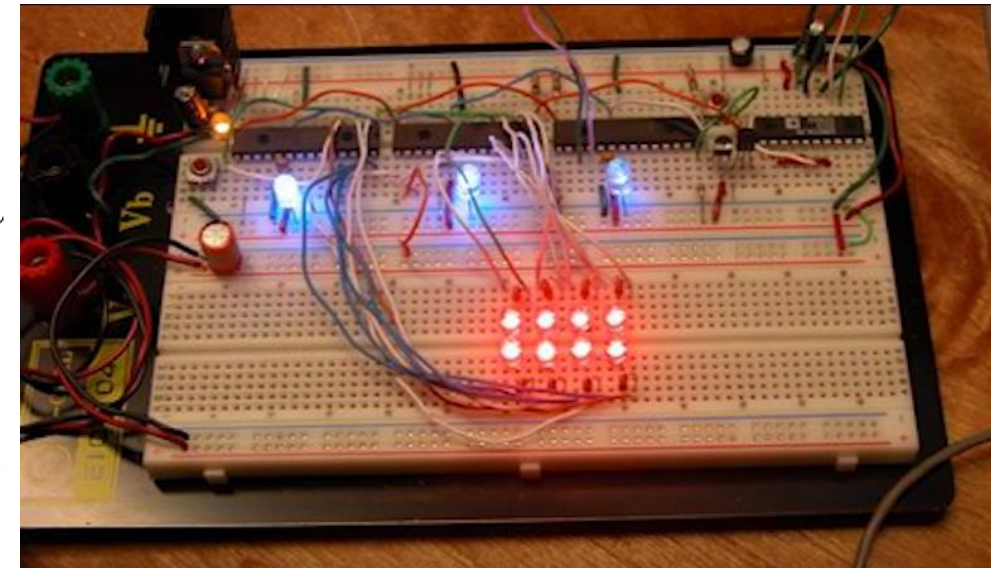
$\downarrow \#1$

$\bar{A}\bar{B}C + 0 + \bar{A}\bar{B}C$

Structural



Physical



CSCE 611

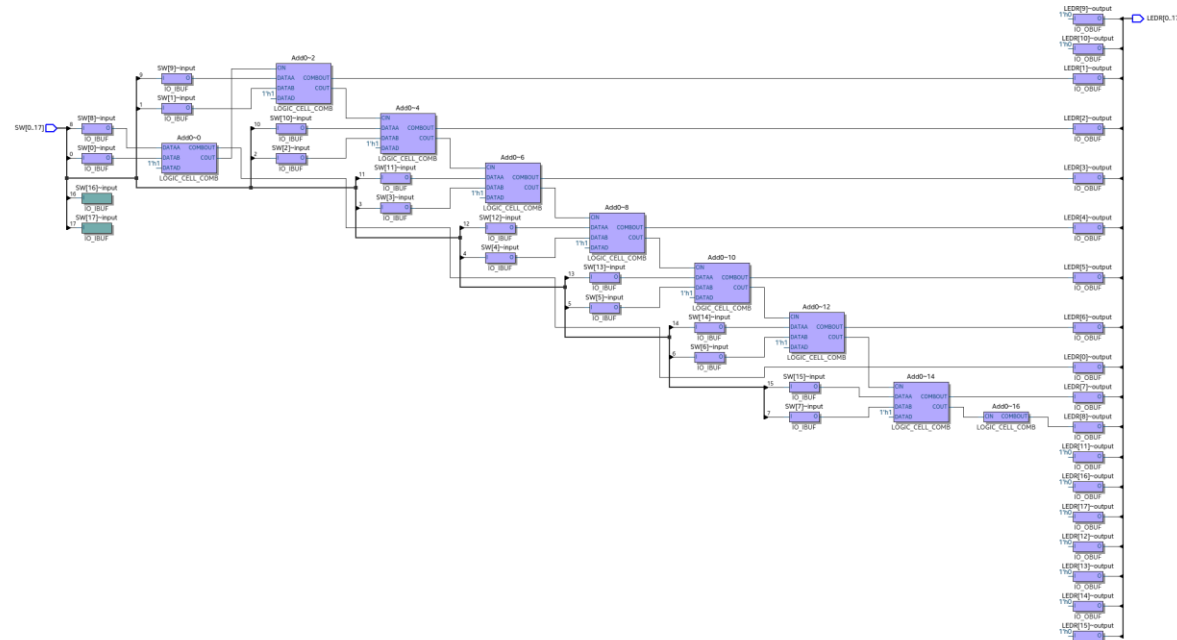
Behavioral

```
module adder ( input logic [17:0] SW,
               output logic [17:0] LEDR );

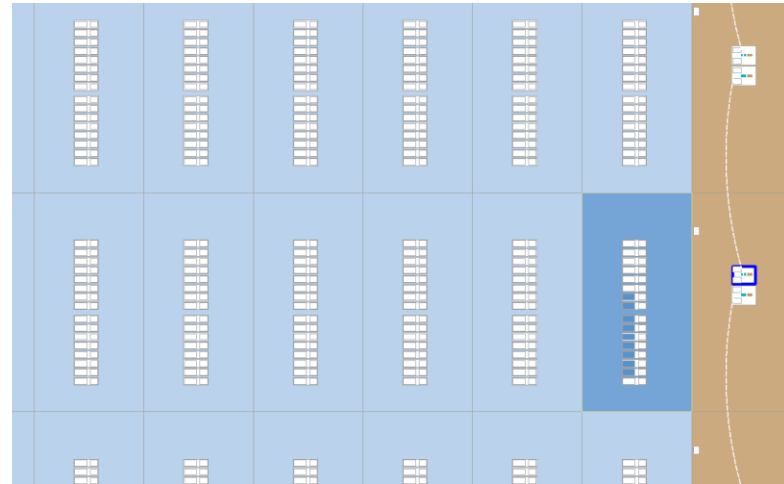
    assign LEDR = SW[7:0] + SW[15:8];

endmodule
```

Structural



Physical



```
module nand3(input logic a, b, c, output logic y);
    logic n1; // internal signal
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3
    inv inverter(.a(n1), .y(y)); // instance of inverter
endmodule
```

Hardware Description Language (HDL)

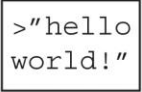
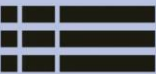
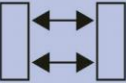
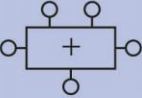
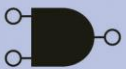
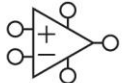
- Describe **behavior** and **structure** in a text-based programming language
 - Use for **simulation** and **synthesis**
- HDLs leverage:
 - **Abstraction**: represent design by hiding details that aren't necessary
 - **Hierarchy**: group design elements into reusable modules
 - **Regularity**: minimize the number of unique modules

Hardware Description Language (HDL)

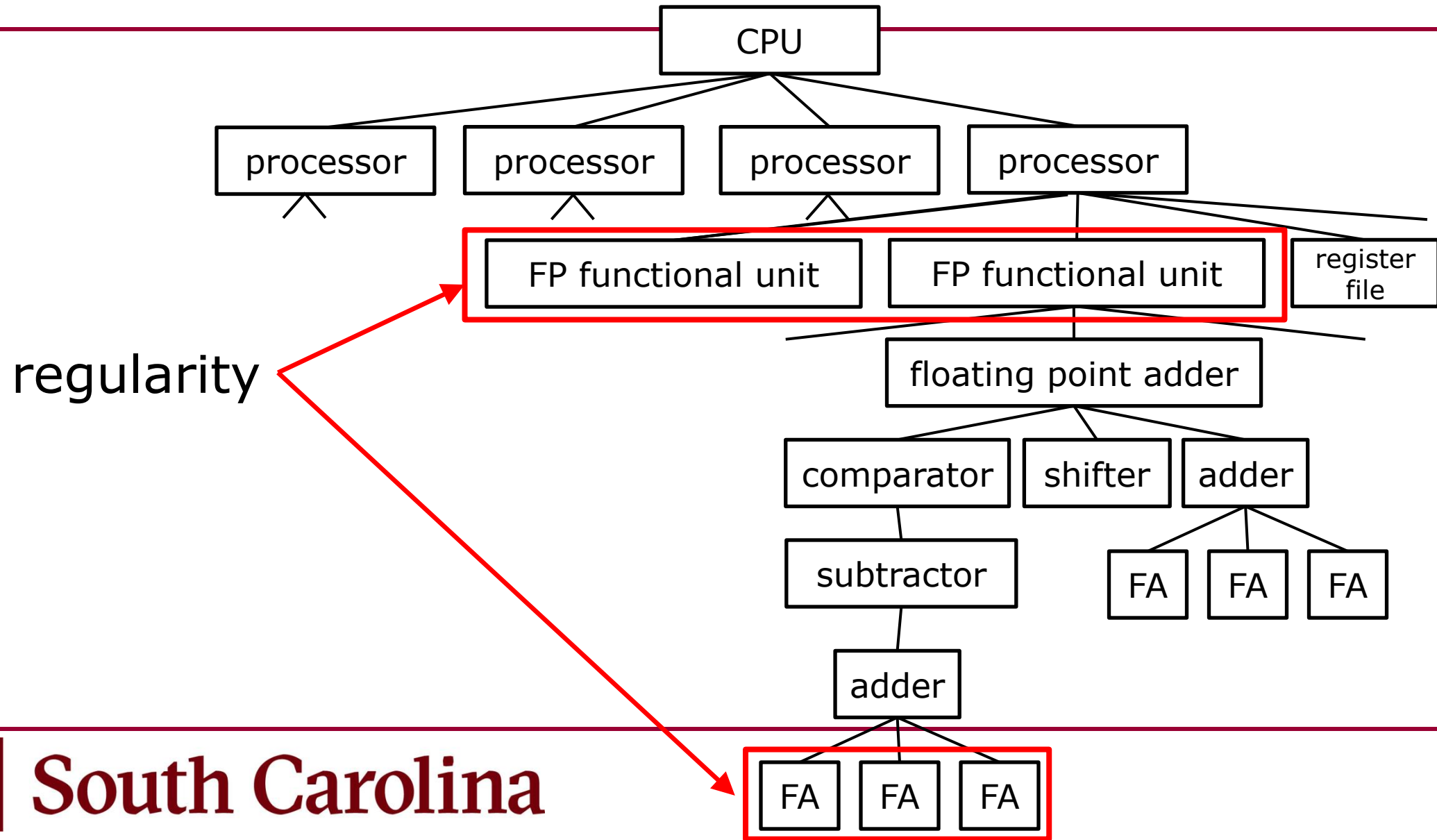
abstraction



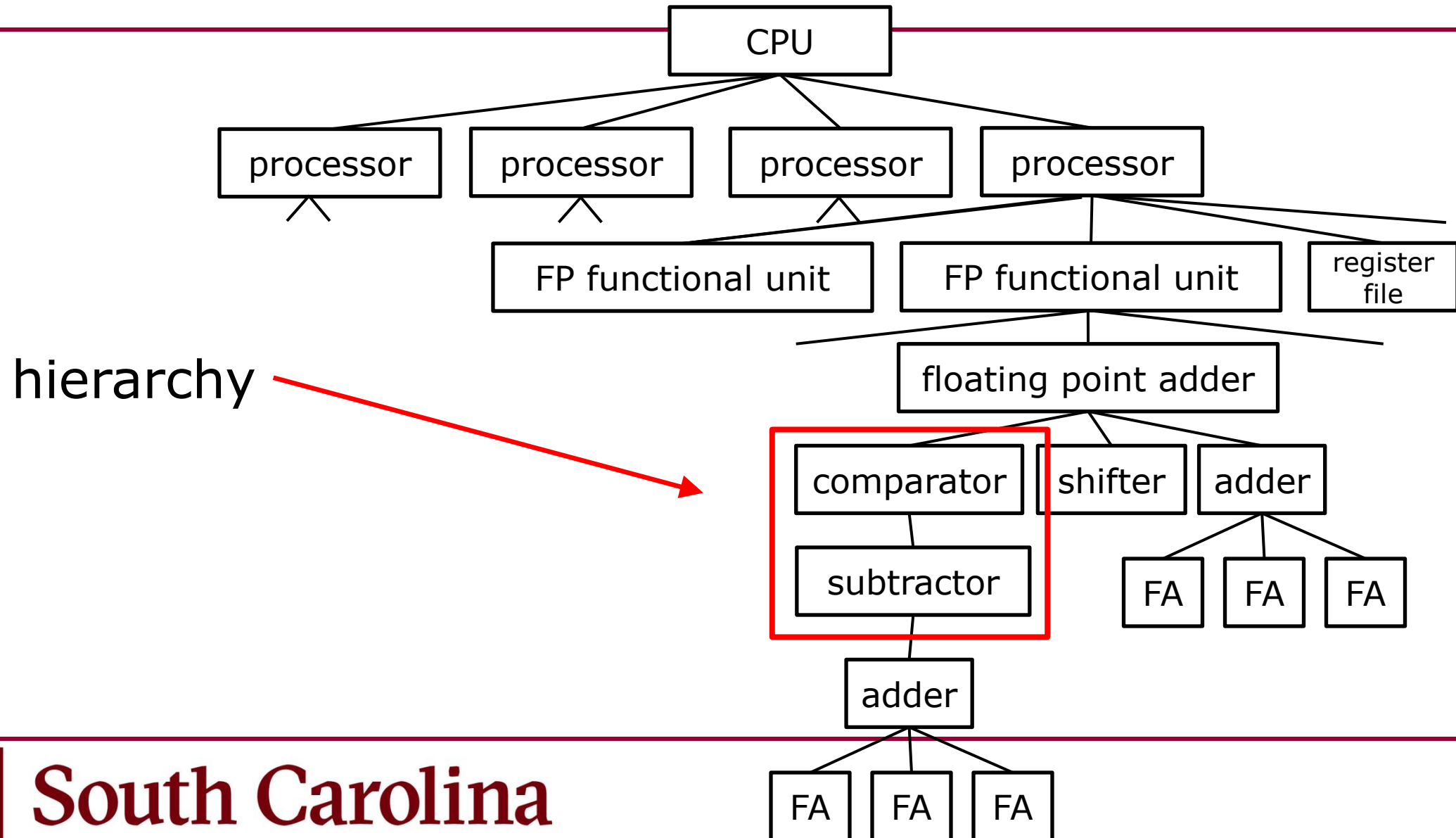
focus of this course

Application Software		programs
Operating Systems		device drivers
Architecture		instructions registers
Micro-architecture		datapaths controllers
Logic		adders memories
Digital Circuits		AND gates NOT gates
Analog Circuits		amplifiers filters
Devices		transistors diodes
Physics		charge carriers

Design Hierarchy

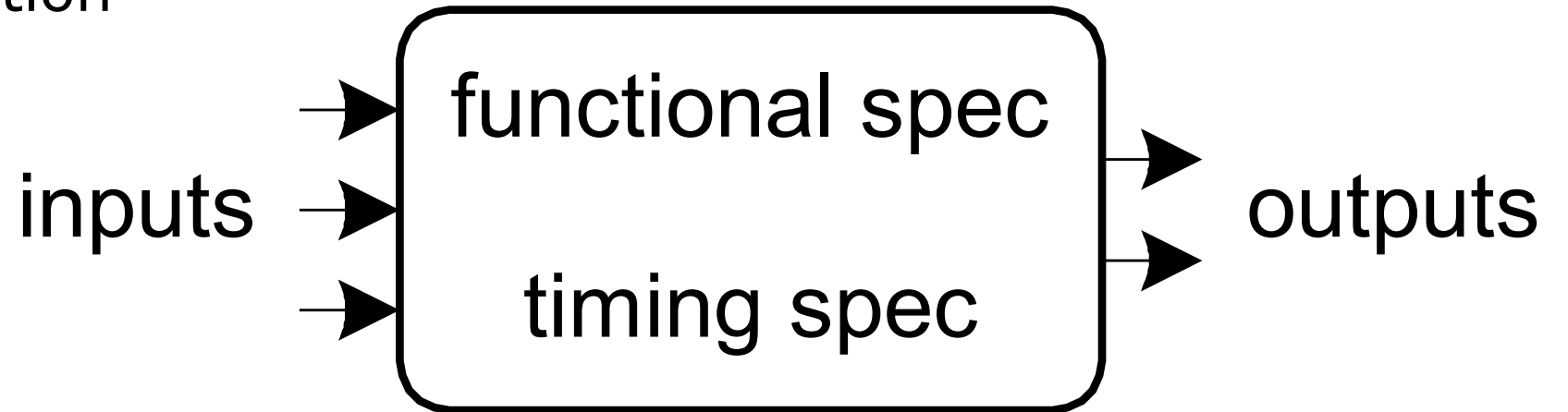


Design Hierarchy



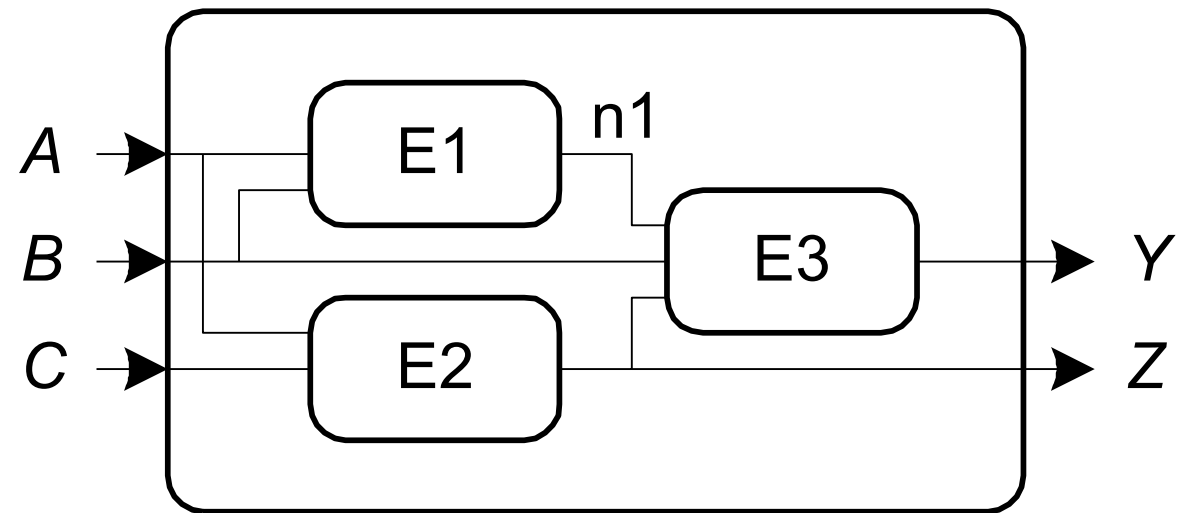
Logic Circuit

- A logic circuit is composed of:
 - inputs
 - outputs
 - functional specification
 - timing specification



Logic Circuit

- Nodes
 - Inputs: A, B, C
 - Outputs: Y, Z
 - Internal: $n1$
- Circuit elements
 - $E1, E2, E3$
 - Each a circuit



Types of Logic Circuits

- **Combinational Logic**

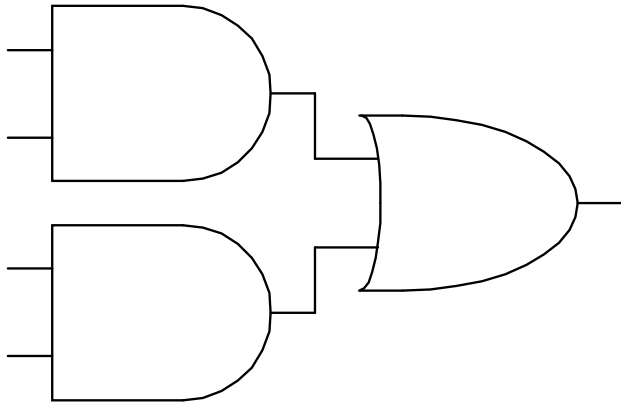
- Memoryless
- Outputs determined by current values of inputs

- **Sequential Logic**

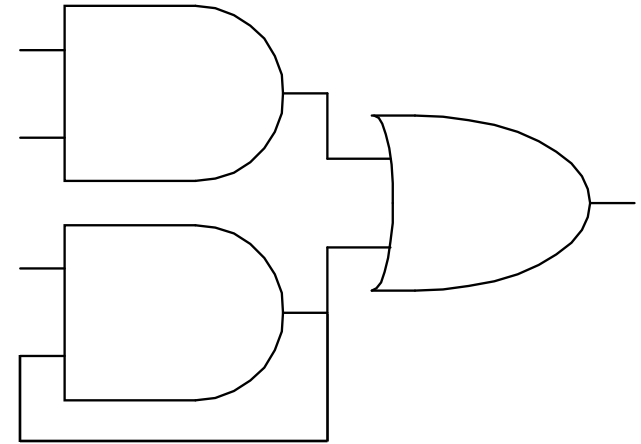
- Has memory
- Outputs determined by previous and current values of inputs

Rules of **Combinational** Composition

- Every element is combinational
- The circuit contains no cyclic paths
- **Example:**



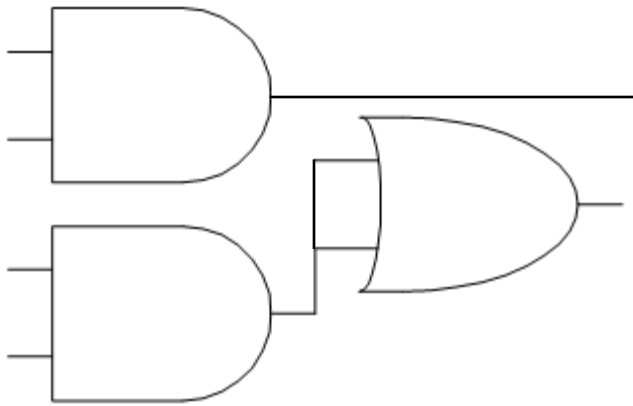
yes



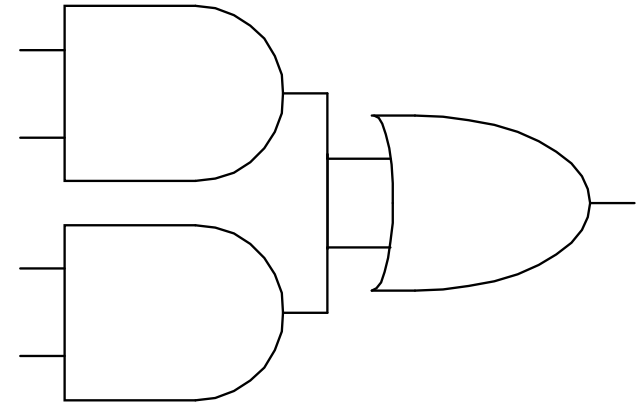
no

Rules of **Combinational** Composition

- Every element is combinational
- Outputs can be connected to $\geq$ input but no other active outputs
- **Example:**



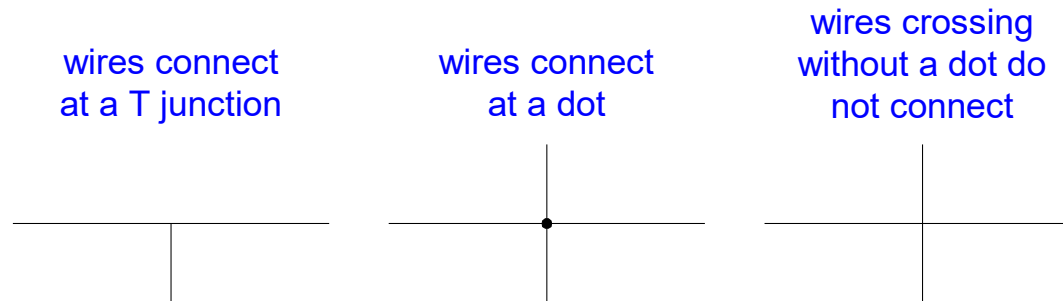
yes



no

Structural: Circuit Schematics Rules

- Inputs on the left (or top)
- Outputs on right (or bottom)
- Gates flow from left to right
- Straight wires are best
- Wires always connect at a T junction
- A dot where wires cross indicates a connection between the wires
- Wires crossing *without* a dot make no connection

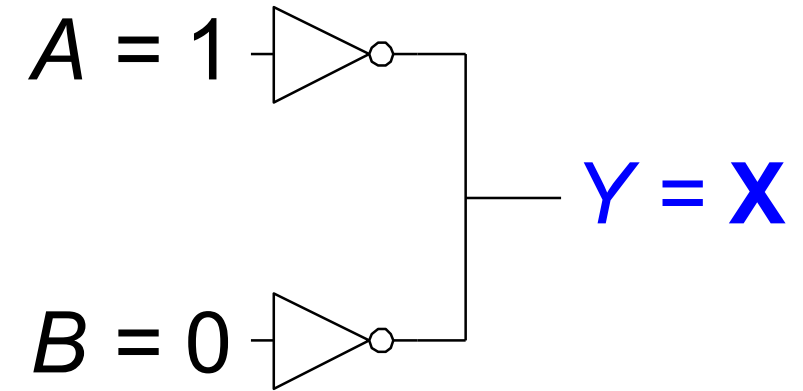


The "logic" Type

- Meant to represent a bit
 - Also for "uninitialized memory bit", "conflict", "don't care", and "undriven"
- "uninitialized memory bit", "conflict", "don't care" all map to value X
- "undriven" maps to value Z
- Thus, logic is an enumeration {1,0,X,Z}

Contention: X

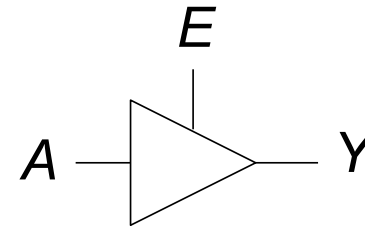
- Contention: circuit tries to drive output to 1 **and** 0
 - Actual value somewhere in between
 - Could be 0, 1, or in forbidden zone
 - Might change with voltage, temperature, time, noise
 - Often causes excessive power dissipation
- **Warnings:**
 - Contention usually indicates a **bug**.
 - **X is used for "don't care", uninitialized memory, and contention** - look at the context to tell them apart
- Verilog offers 8 levels of "strength" to resolve conflicts (for simulation only), but in this class we're using the default strength of strong



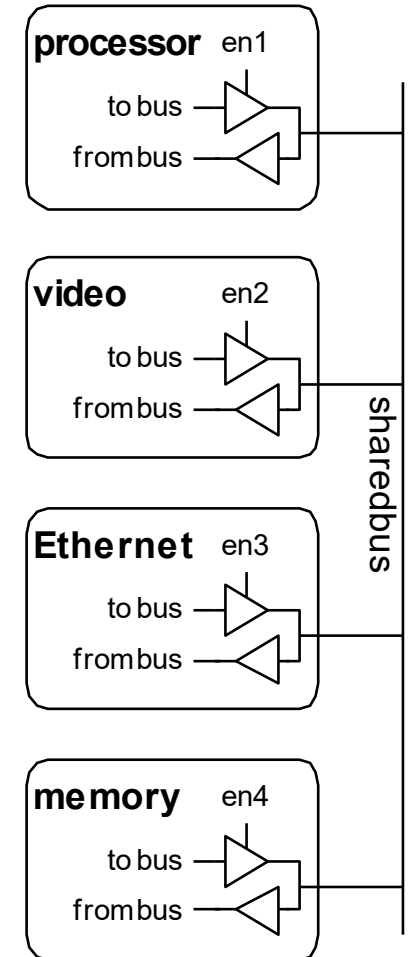
Floating: Z

- Floating, high impedance, open, high Z
- Floating output might be 0, 1, or somewhere in between
 - A voltmeter won't indicate whether a node is floating
- Floating nodes are used in tristate busses
 - Many different drivers
 - Exactly one is active at once

Tristate Buffer



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



Hardware Description Language

- Hardware description language (HDL):
 - Specifies logic function only
 - Computer-aided design (CAD) tool produces or *synthesizes* the optimized gates
- Most commercial designs built using HDLs
- Two leading HDLs:
 - **VHDL (VHDL 2008)**
 - VHSIC (“Very High Speed Integrated Circuit”) HDL
 - Developed in 1981 by the Department of Defense to specify the behavior of custom chips
 - Simulators and synthesizers followed
 - Became an IEEE standard (1076) in 1987
 - Updated in 2008 (IEEE STD 1076-2008)
 - Based on ADA (looks like Pascal), designed to be verbose and easy-to-read
 - **Verilog (and SystemVerilog)**
 - Developed in 1984 by Gateway Design Automation
 - IEEE standard (1364) in 1995
 - Extended in 2005 (IEEE STD 1800-2009)

HDL to Gates

- **Simulation**

- Inputs applied to circuit
- Outputs checked for correctness
- Millions of dollars saved by debugging in simulation instead of hardware

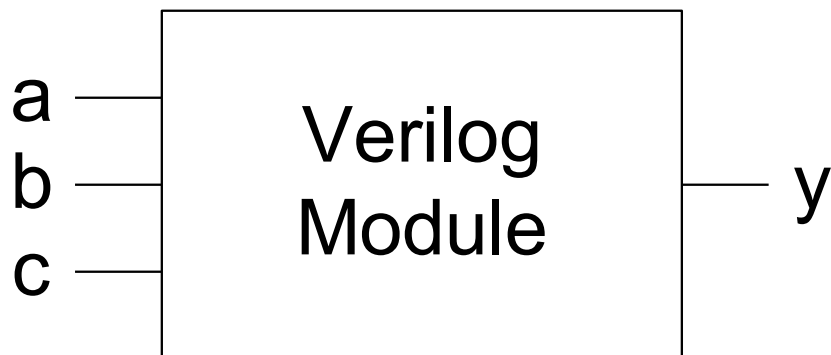
- **Synthesis**

- Transforms HDL code into a *netlist* describing the hardware (i.e., a list of gates and the wires connecting them)

IMPORTANT:

When using an HDL, think of the **hardware** the HDL should produce

SystemVerilog Modules



Two types of code "styles":

- **Behavioral:** describe what a module does
- **Structural:** describe how it is built from simpler modules

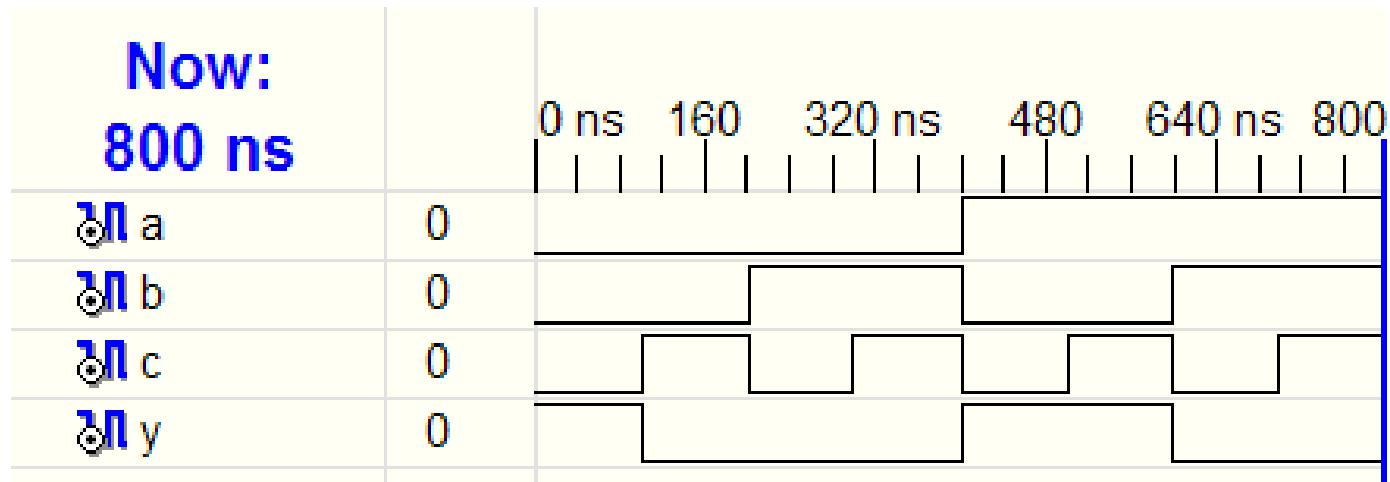
- modules leverage:
 - **Hierarchy:** for a specific design
 - **Regularity:** minimize the complexity (number of unique modules) of the underlying level

Behavioral Verilog Example

```
module example(input logic a, b, c,  
               output logic y);  
    assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;  
endmodule
```

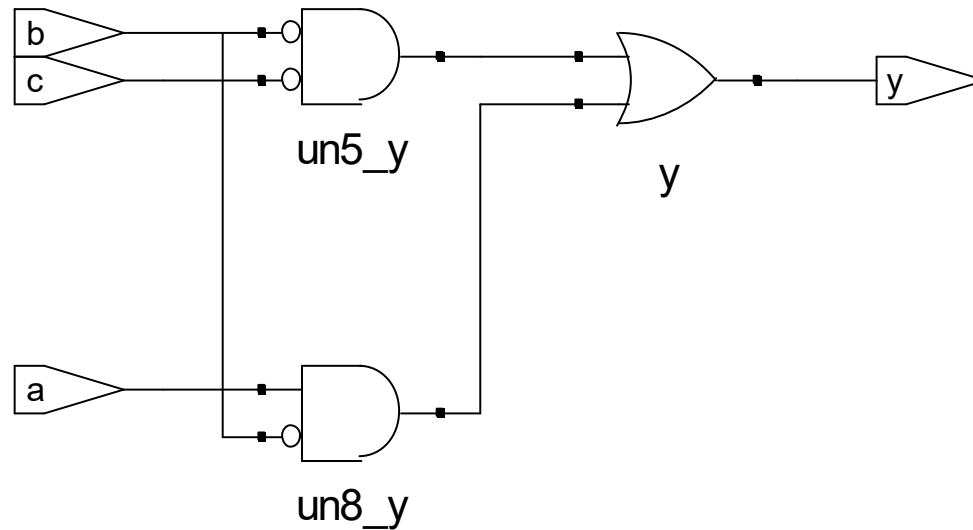
Behavioral Verilog Example

```
module example(input logic a, b, c,  
               output logic y);  
    assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;  
endmodule
```



Behavioral Verilog Example

```
module example(input logic a, b, c,  
               output logic y);  
    assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;  
endmodule
```



SystemVerilog Syntax

- Case sensitive
 - Example: reset and Reset are not the same signal
- No names that start with numbers
 - Example: 2mux is an invalid name
- Whitespace ignored
- Comments:
 - // single line comment
 - /* multiline
comment */

Ports

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule
```

```
module mydesign();  
    logic a,b,c,n1;  
    and3 andgate(.y(n1), .b(b), .a(a), .c(c));  
endmodule
```

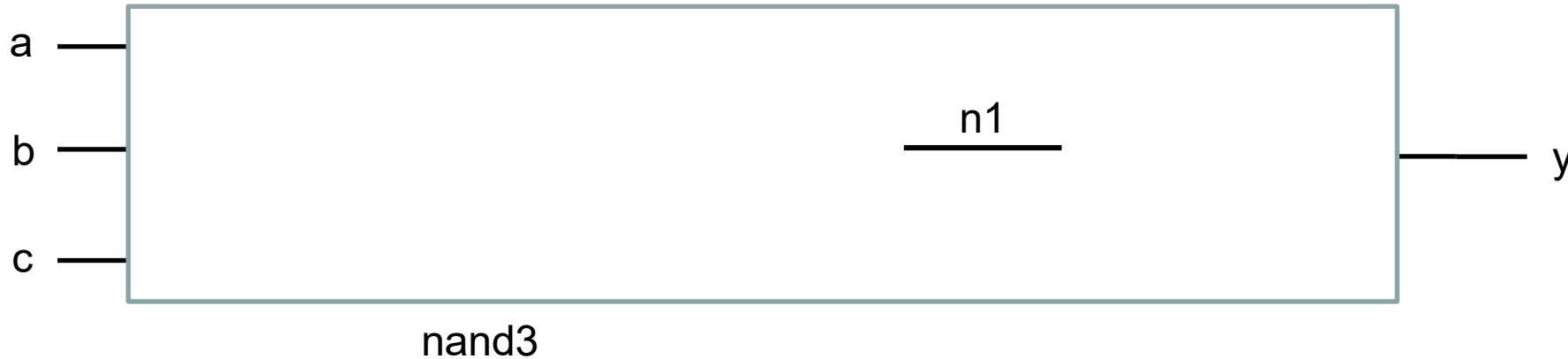
Structural Modeling - Hierarchy

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule  
  
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule  
  
module nand3(input logic a, b, c, output logic y);  
    logic n1; // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y)); // instance of inverter  
endmodule
```



Structural Modeling - Hierarchy

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule  
  
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule  
  
module nand3(input logic a, b, c, output logic y);  
    logic n1; // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y)); // instance of inverter  
endmodule
```

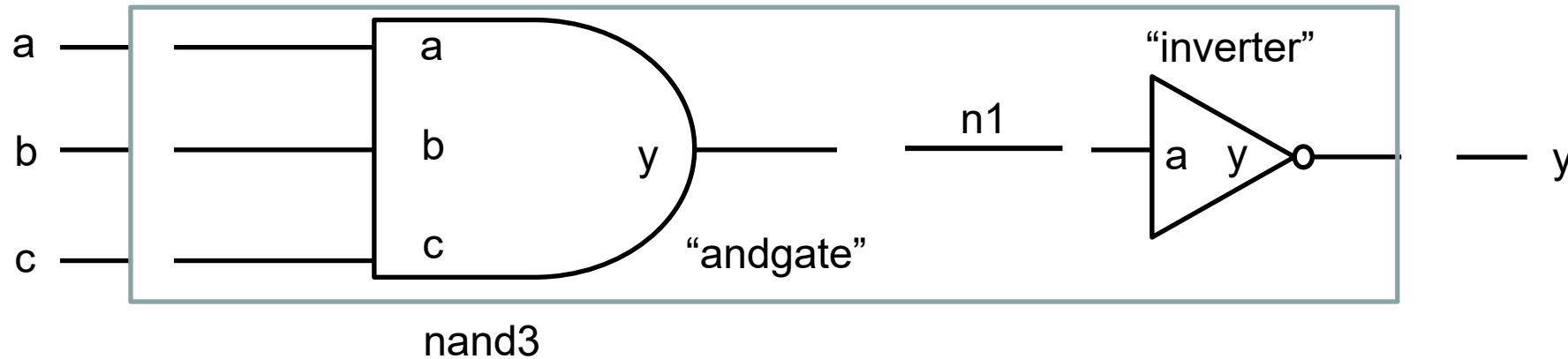


Structural Modeling - Hierarchy

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule
```

```
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule
```

```
module nand3(input logic a, b, c, output logic y);  
    logic n1; // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y)); // instance of inverter  
endmodule
```

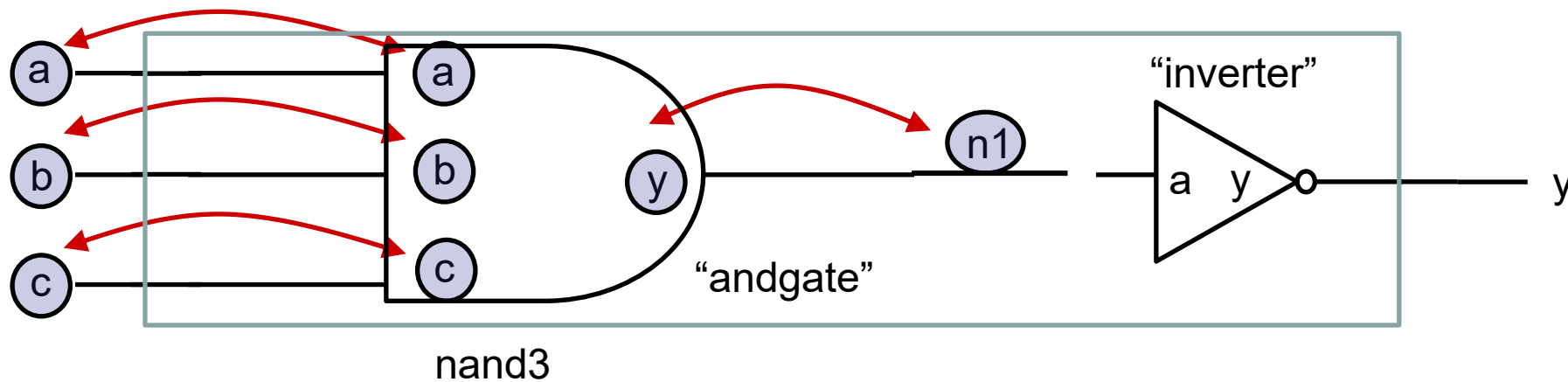


Structural Modeling - Hierarchy

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule
```

```
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule
```

```
module nand3(input logic a, b, c, output logic y);  
    logic n1; // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y)); // instance of inverter  
endmodule
```

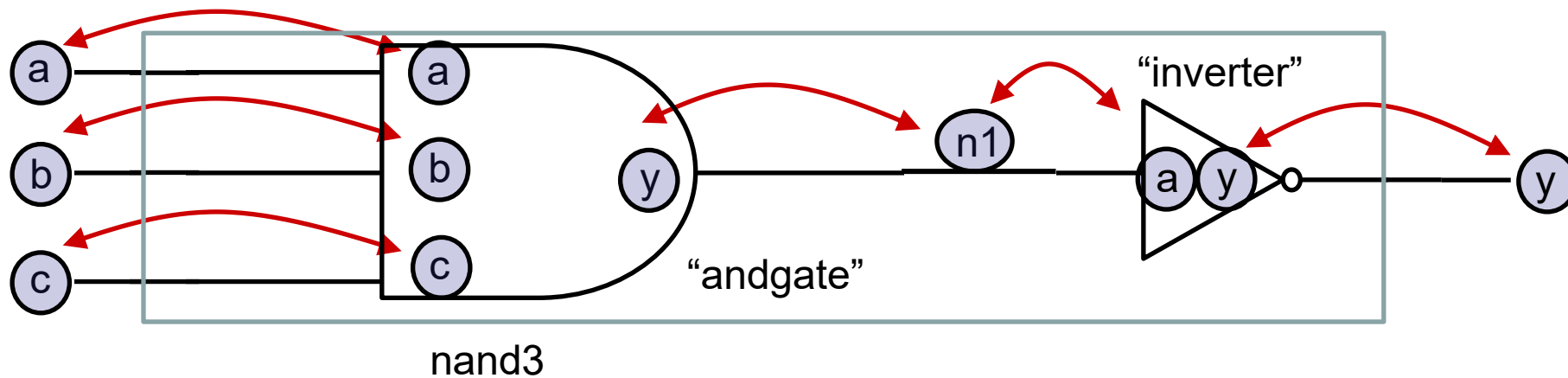


Structural Modeling - Hierarchy

```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule
```

```
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule
```

```
module nand3(input logic a, b, c, output logic y);  
    logic n1; // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y)); // instance of inverter  
endmodule
```



Numbers

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hexadecimal	171	10101011
42	unsized	decimal	42	00...0101010

Structural Modeling - Hierarchy

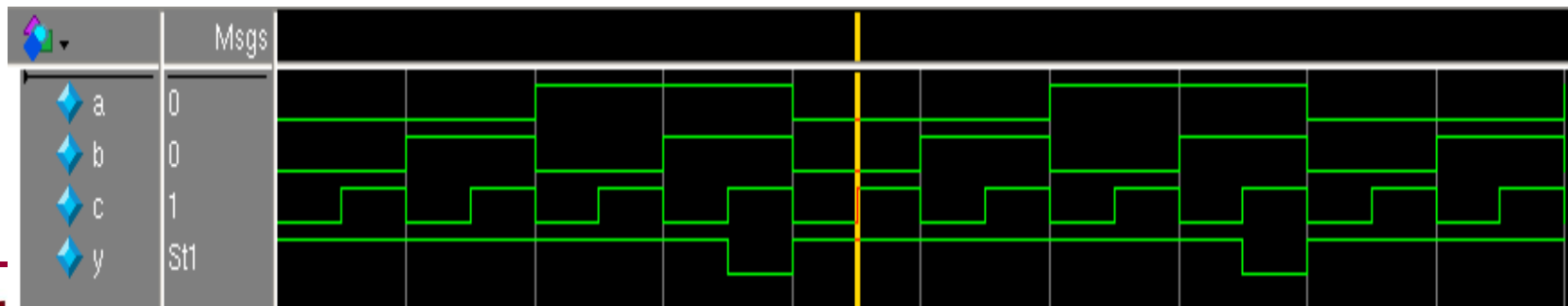
```
module and3(input logic a, b, c, output logic y);  
    assign y = a & b & c;  
endmodule
```

module I/O (ports)

```
module inv(input logic a, output logic y);  
    assign y = ~a;  
endmodule
```

port map

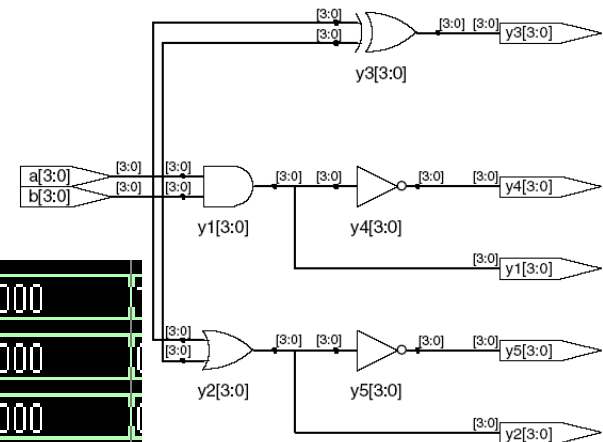
```
module nand3(input logic a, b, c, output logic y);  
    logic n1;                                // internal signal  
    and3 andgate(.a(a), .b(b), .c(c), .y(n1)); // instance of and3  
    inv inverter(.a(n1), .y(y));              // instance of inverter  
endmodule
```



Bitwise Operators

```
module gates(input logic [3:0] a, b,  
             output logic [3:0] y1, y2, y3, y4, y5);  
    /* Five different two-input logic  
       gates acting on 4 bit busses */  
    assign y1 = a & b;      // AND  
    assign y2 = a | b;      // OR  
    assign y3 = a ^ b;      // XOR  
    assign y4 = ~(a & b);   // NAND  
    assign y5 = ~(a | b);   // NOR  
endmodule
```

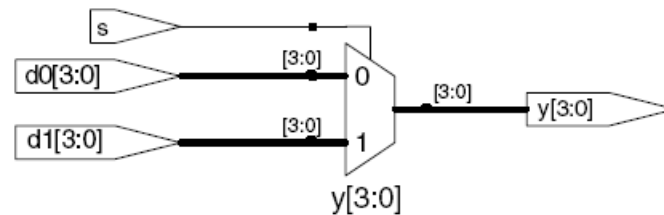
order of
statements
doesn't matter!!!



a	0100	0000	0001	0010	0011	0100	0101	0110	0111	1000
b	1100	0000	1111	1110	1101	1100	1011	1010	1001	1000
y1	0100	0000	0001	0010	0001	0100	0001	0010	0001	1000
y2	1100	0000	1111	1110	1111	1100	1111	1110	1111	1000
y3	1000	0000	1110	1100	1110	1000	1110	1100	1110	0000
y4	1011	1111	1110	1101	1110	1011	1110	1101	1110	0111
y5	0011	1111	0000	0001	0000	0011	0000	0001	0000	0111

Conditional Assignment

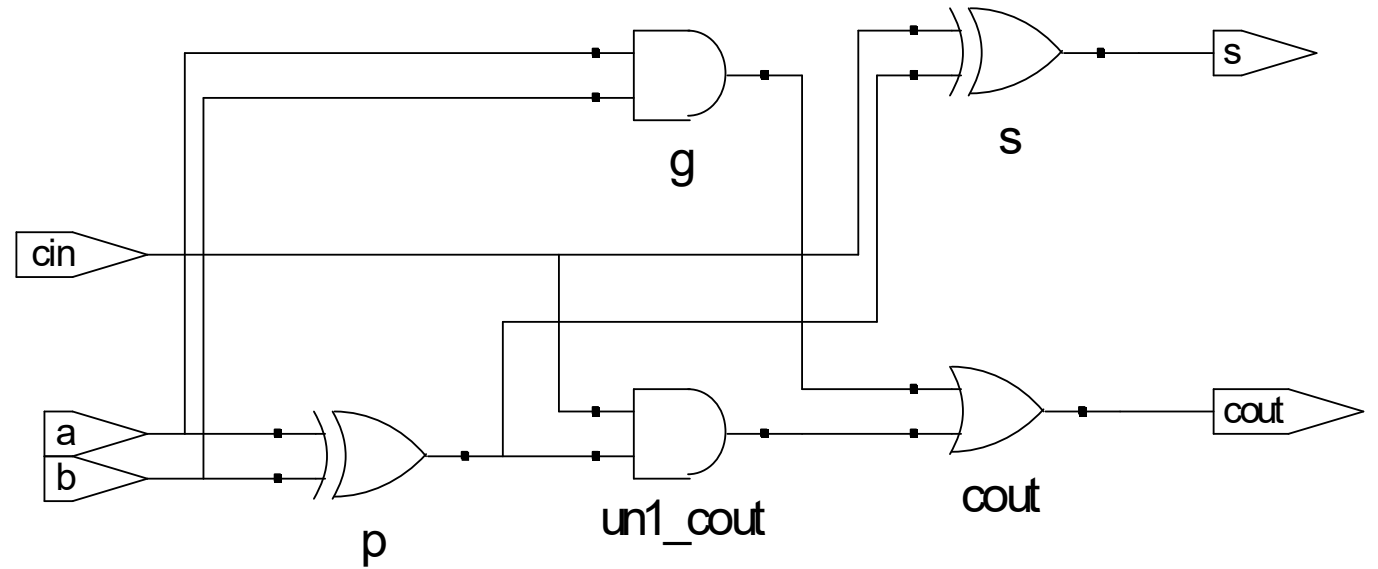
```
module mux2(input logic [3:0] d0, d1,  
            input logic s,  
            output logic [3:0] y);  
    assign y = s ? d1 : d0;  
endmodule
```



		d0	0010	0000	0001	0010	0011	0100
		d1	1110	0000	1111	1110	1101	1100
		s	1					
		y	1110	0000	0001	1110	1101	0100

Internal Signals

```
module fulladder(input logic a, b, cin,  
                 output logic s, cout);  
    logic p, g; // internal nodes  
  
    assign p = a ^ b;  
    assign g = a & b;  
  
    assign s = p ^ cin;  
    assign cout = g | (p & cin);  
endmodule
```



Precedence

Highest

~	NOT
*, /, %	mult, div, mod
+, -	add, sub
<<, >>	shift
<<<, >>>	arithmetic shift
<, <=, >, >=	comparison
==, !=	equal, not equal
&, ~&	AND, NAND
^, ~^	XOR, XNOR
, ~	OR, NOR
? :	ternary operator

Lowest

Bit Manipulations: Example 1

```
assign y = {a[2:1], {3{b[0]}}}, a[0], 6'b100_010};  
// if y is a 12-bit signal, the above statement produces:  
// y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0
```

or:

```
assign {a[2:1], b[0]} = y;
```

```
// underscores (_) are used for formatting only to make  
// it easier to read. SystemVerilog ignores them.
```

Bit Manipulations: Example 1

- Example: sign extend b[15:0]

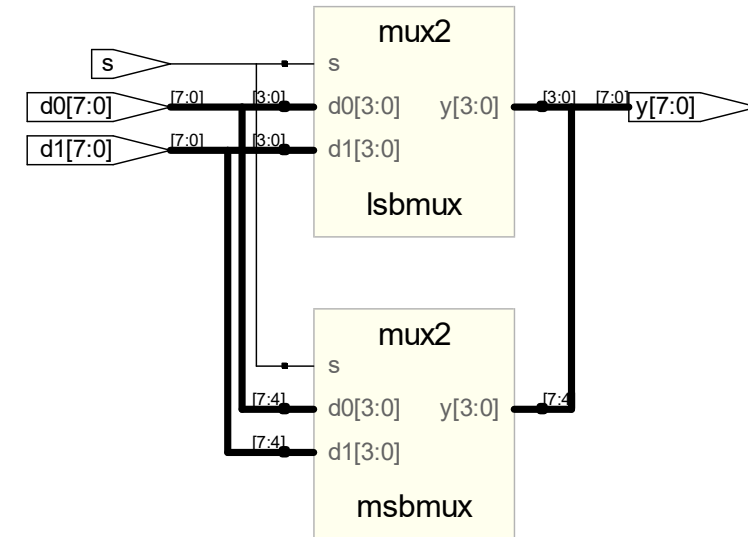
```
assign b_ext = {b[15], b[15], b[15], b[15], b[15], b[15], b[15], b[15], b[15],  
b[15], b[15], b[15], b[15], b[15], b[15], b};
```

– or:

```
assign b_ext = { {16{b[15]}} , b };
```

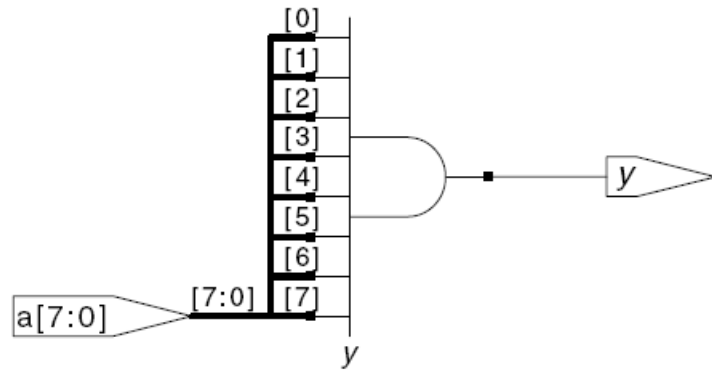
Bit Manipulations: Example 2

```
module mux2_8(input  [7:0] d0, d1,  
              input      s,  
              output [7:0] y);  
  
    mux2 lsbmux(.a(d0[3:0]), .b(d1[3:0]), .sel(s), .out(y[3:0]));  
    mux2 msbmux(.a(d0[7:4]), .b(d1[7:4]), .sel(s), .out(y[7:4]));  
endmodule
```



Reduction Operators

```
module and8(input  [7:0] a,  
            output y);  
    assign y = &a;  
    // &a is much easier to write than  
    // assign y = a[7] & a[6] & a[5] & a[4] &  
    //           a[3] & a[2] & a[1] & a[0];  
endmodule
```



Available reduction operators:

- & (and)
- ~& (nand)
- | (or)
- ~| (nor)
- ^ (xor)
- ~^ (xnor)

Logic with Ternary Operator

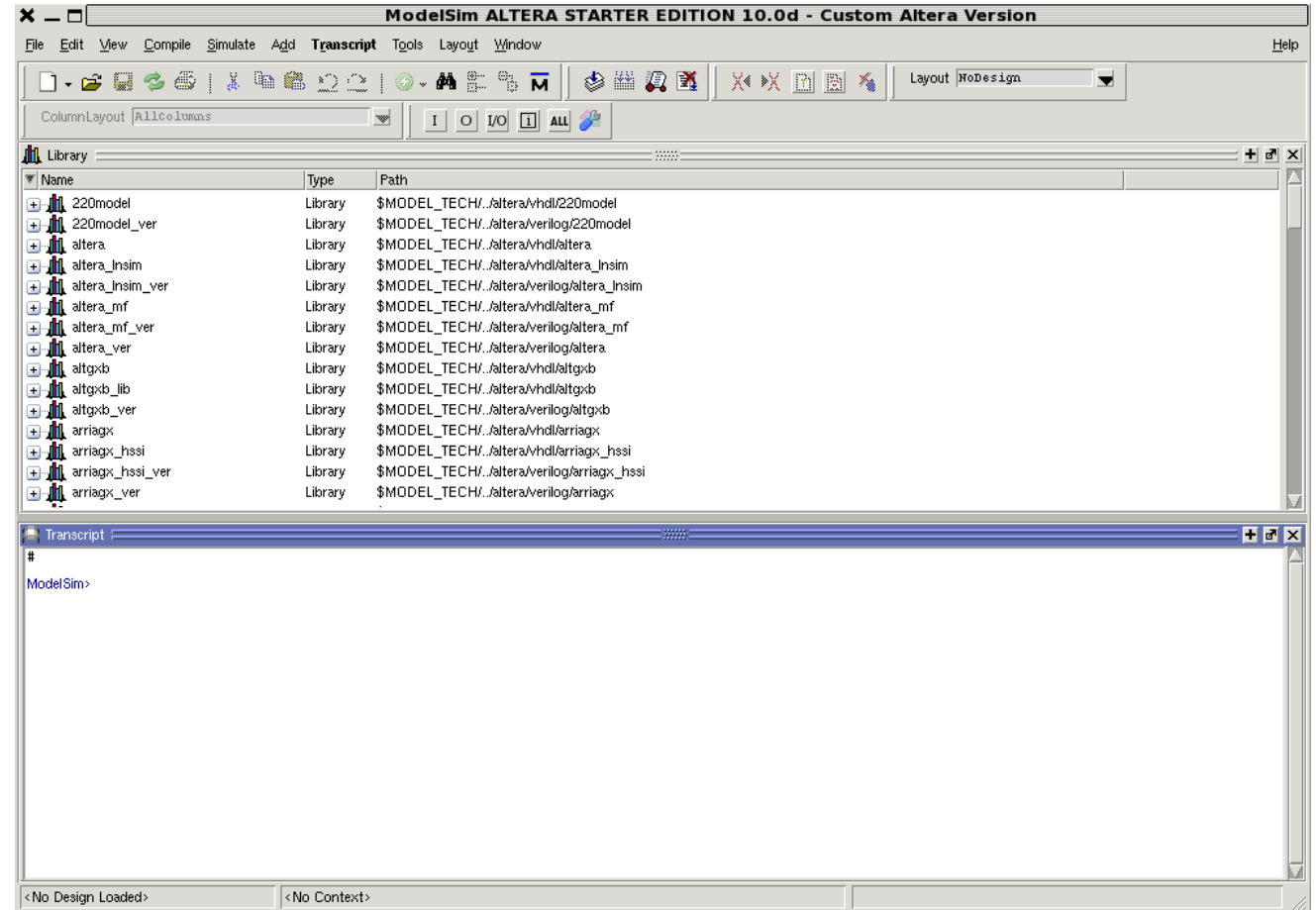
```
assign HEX_out = input_bits == 4'd0 ? 7'b1011100 :  
                  input_bits == 4'd1 ? 7'b1100000 :  
                  7'b1000000;
```

Modelsim

- To start GUI, in terminal window:

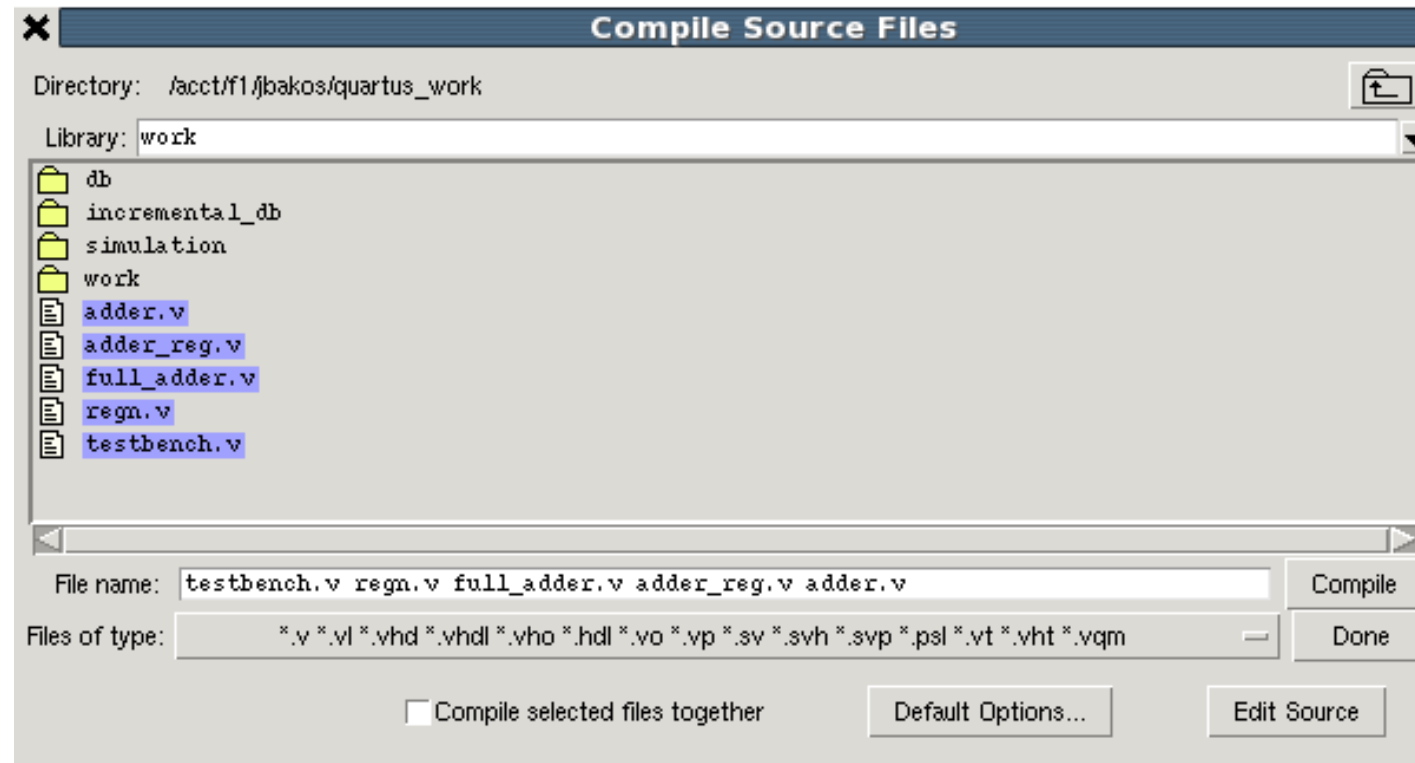
vsim&

No design currently
being simulated



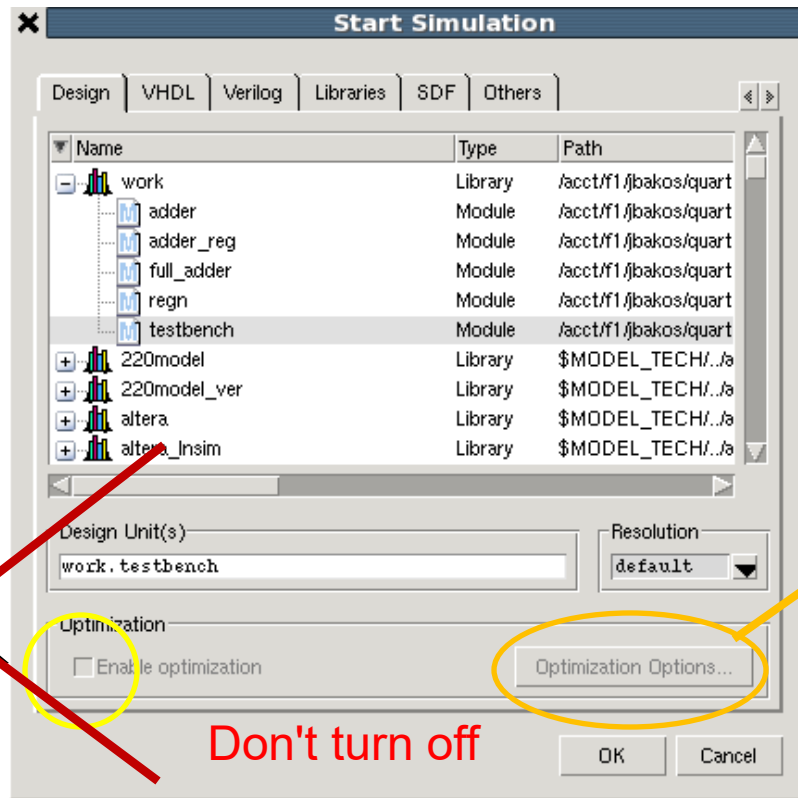
Modelsim

- Select Compile | Compile...
- Select all five Verilog files for our design and click Compile then Done

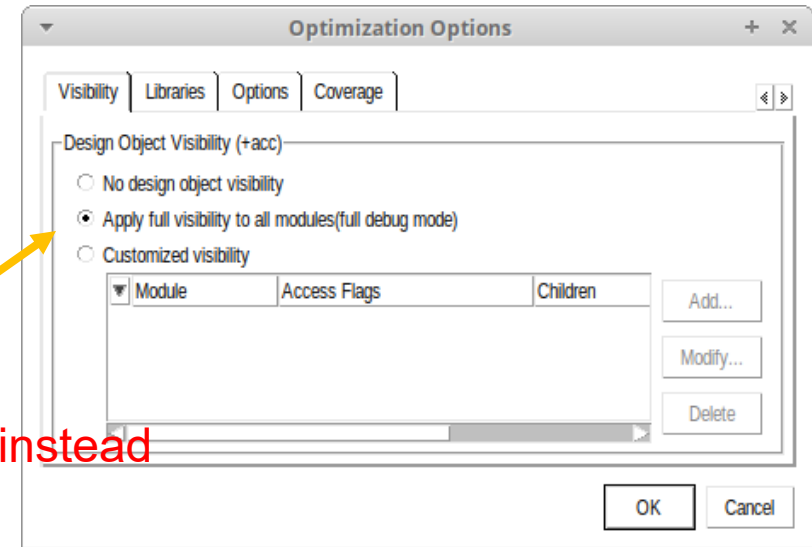


Modelsim

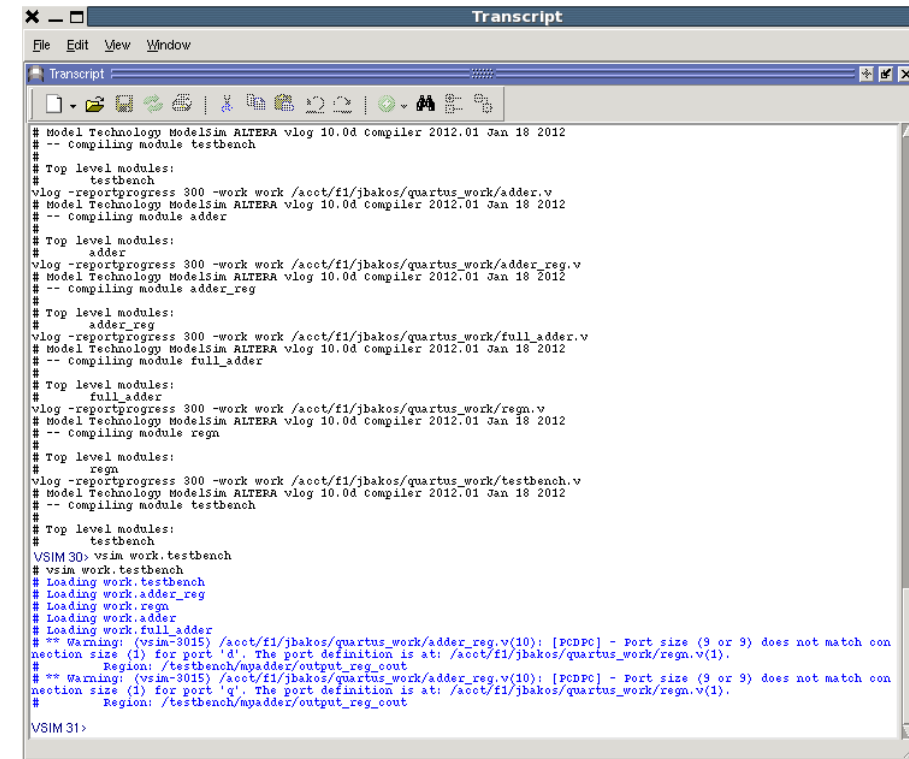
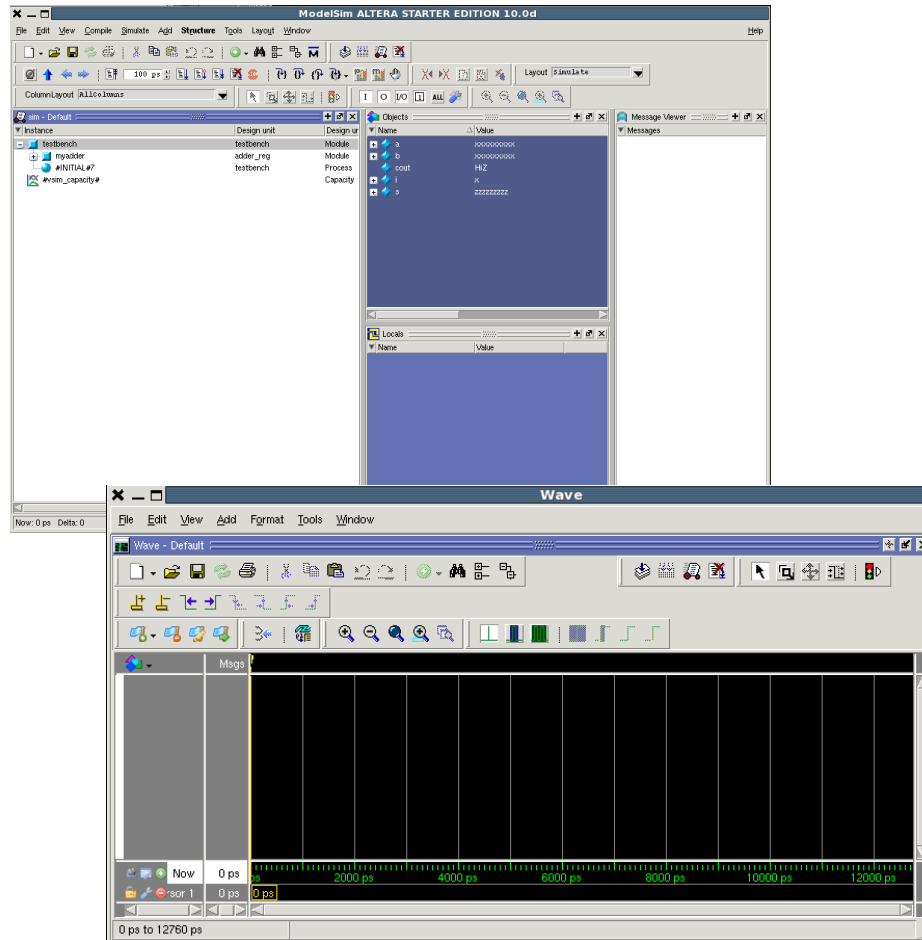
- Now select Simulate | Start Simulation
- Under the work library select testbench and click Ok



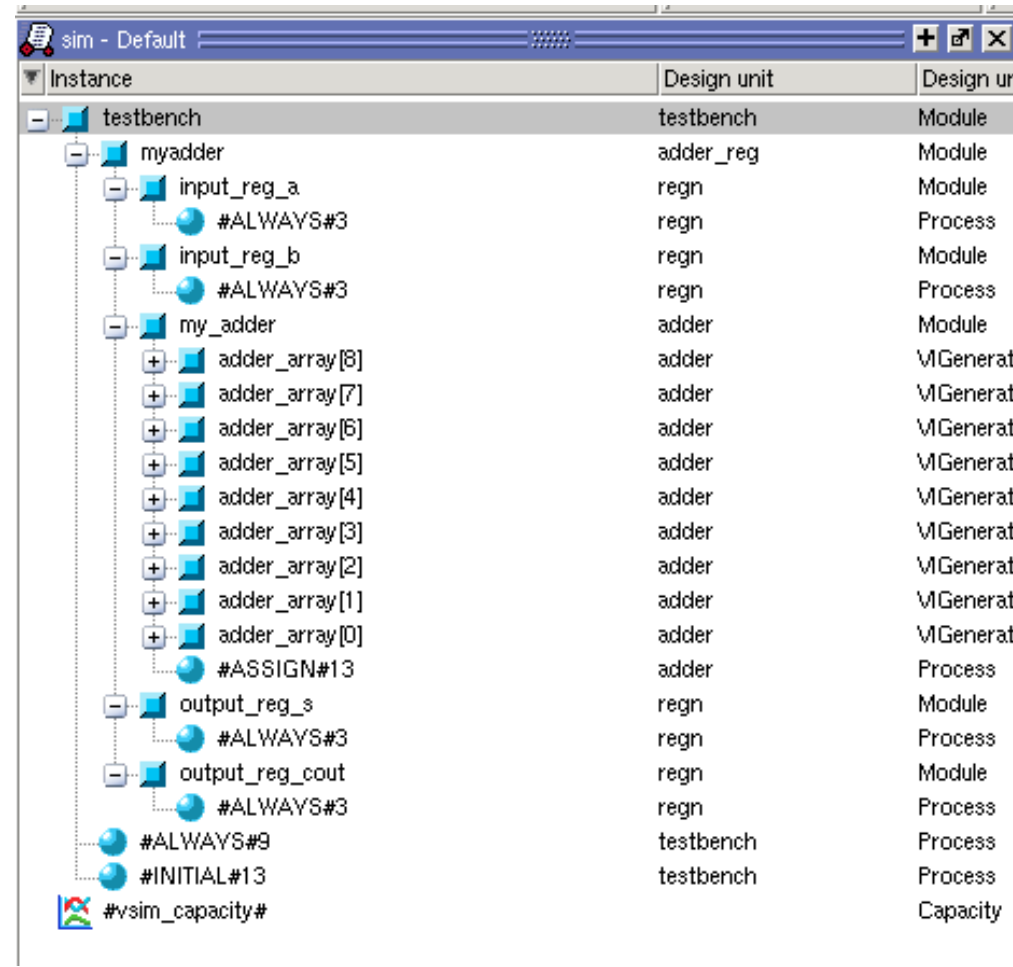
Do this instead



Modelsim Windows



Design Structure

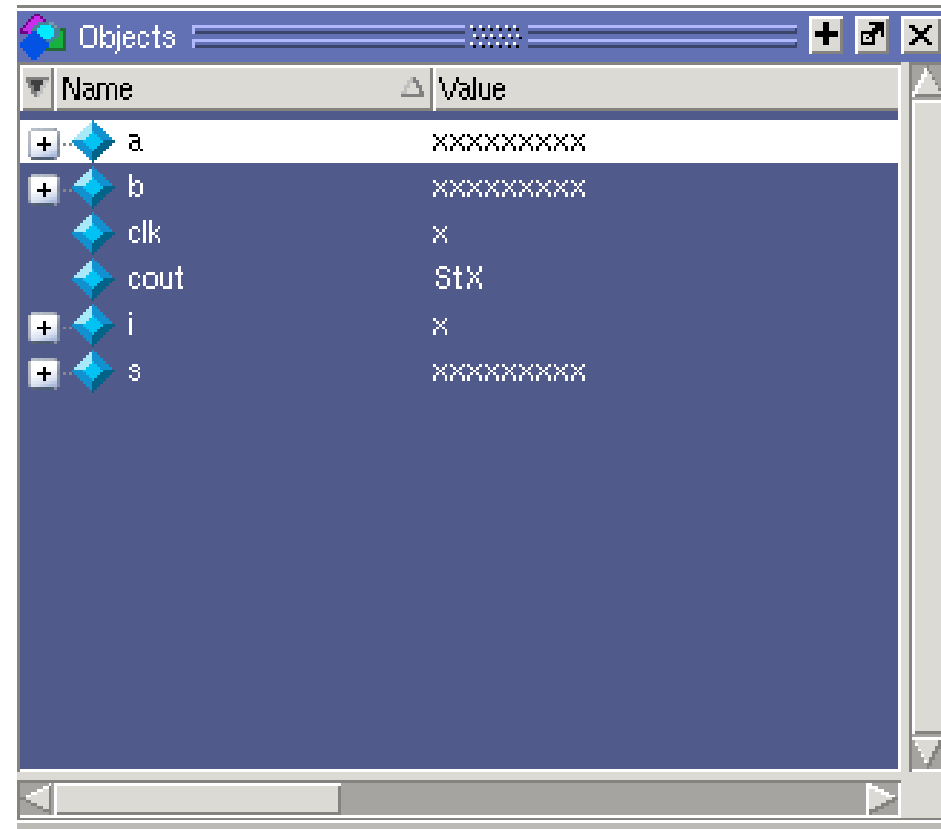


The screenshot shows a simulation tool window titled "sim - Default". On the left is a hierarchical tree view of the design structure. On the right is a table with three columns: "Instance", "Design unit", and "Design ur".

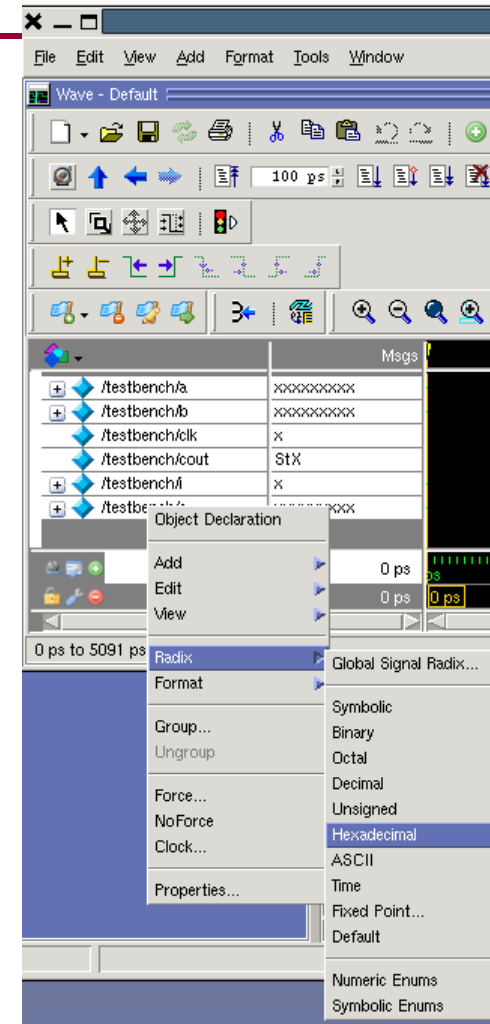
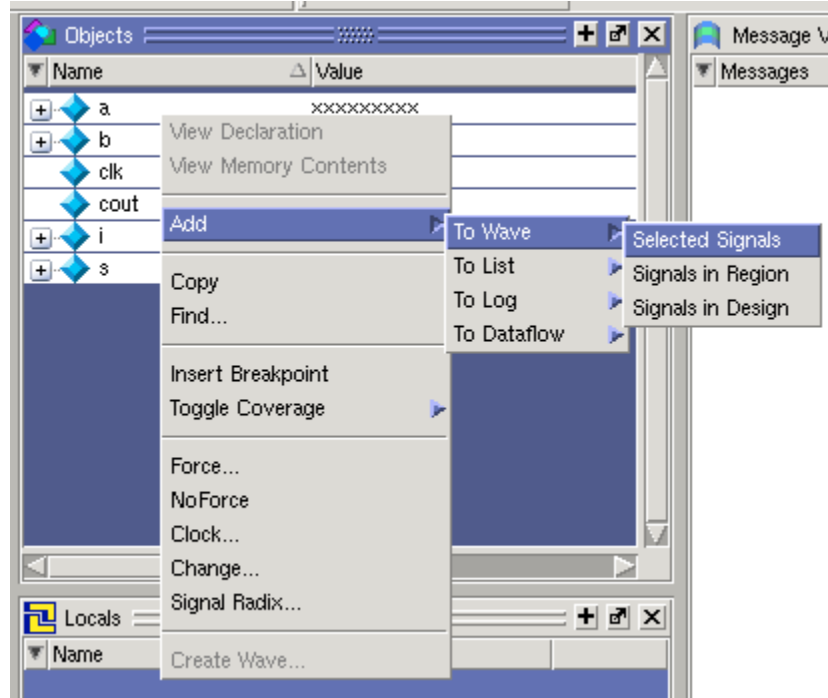
Instance	Design unit	Design ur
testbench	testbench	Module
myadder	adder_reg	Module
input_reg_a	regn	Module
#ALWAYS#3	regn	Process
input_reg_b	regn	Module
#ALWAYS#3	regn	Process
my_adder	adder	Module
adder_array[8]	adder	VGenerat
adder_array[7]	adder	VGenerat
adder_array[6]	adder	VGenerat
adder_array[5]	adder	VGenerat
adder_array[4]	adder	VGenerat
adder_array[3]	adder	VGenerat
adder_array[2]	adder	VGenerat
adder_array[1]	adder	VGenerat
adder_array[0]	adder	VGenerat
#ASSIGN#13	adder	Process
output_reg_s	regn	Module
#ALWAYS#3	regn	Process
output_reg_cout	regn	Module
#ALWAYS#3	regn	Process
#ALWAYS#9	testbench	Process
#INITIAL#13	testbench	Process
#vsim_capacity#		Capacity

Design Objects

- Note: the contents of this pane depend on what is selected in structure pane

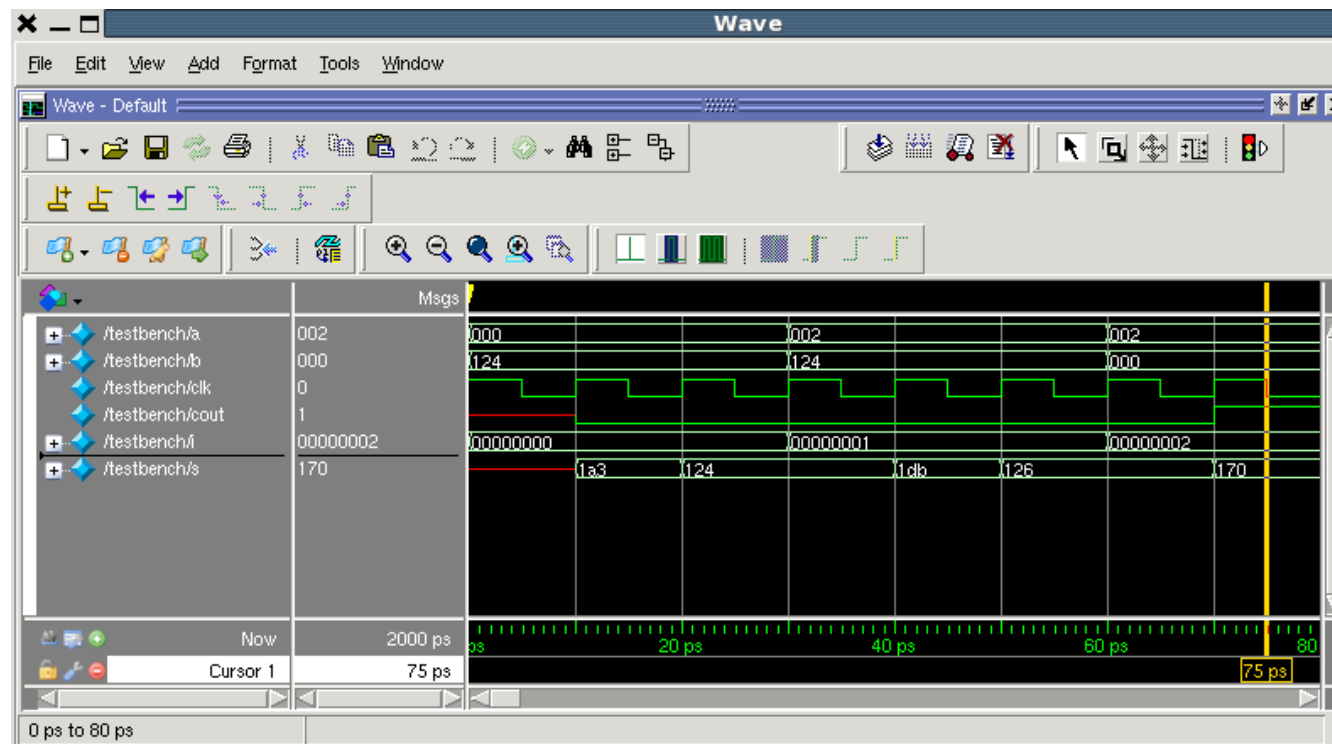


Wave Window



Modelsim

- In transcript window, type "run 30000"



```
VSIM 69> run 30000
# TEST DATA
# 006 + 124 = 12a
# 19b + 000 = 19b
# 1d5 + 002 = 1d7
# 006 + 124 = 12a
# 19b + 000 = 19b
# 1d5 + 002 = 1d7
# 006 + 124 = 12a
# 19b + 000 = 19b
# 1d5 + 002 = 1d7
# 006 + 124 = 12a
# 19b + 000 = 19b
# 1d5 + 002 = 1d7
# 006 + 124 = 12a
# 19b + 000 = 19b
# 1d5 + 002 = 1d7
# 006 + 124 = 12a
# 19b + 000 = 19b
```

Modelsim Notes

- Need to recompile after any changes
- “restart -f” will reload and restart simulation
- Signals must be added to wave or “log” in order to track
- You can add signals in lower levels of hierarchy by “drilling down” in structure and choosing signals in objects window

- Other useful Modelsim commands
 - force A X”DEADBEEF”
 - force op “0001”
 - force push 1
 - run 100
 - force -freeze clk 1 0, 0 {50 ps} -r 100

Modelsim Notes

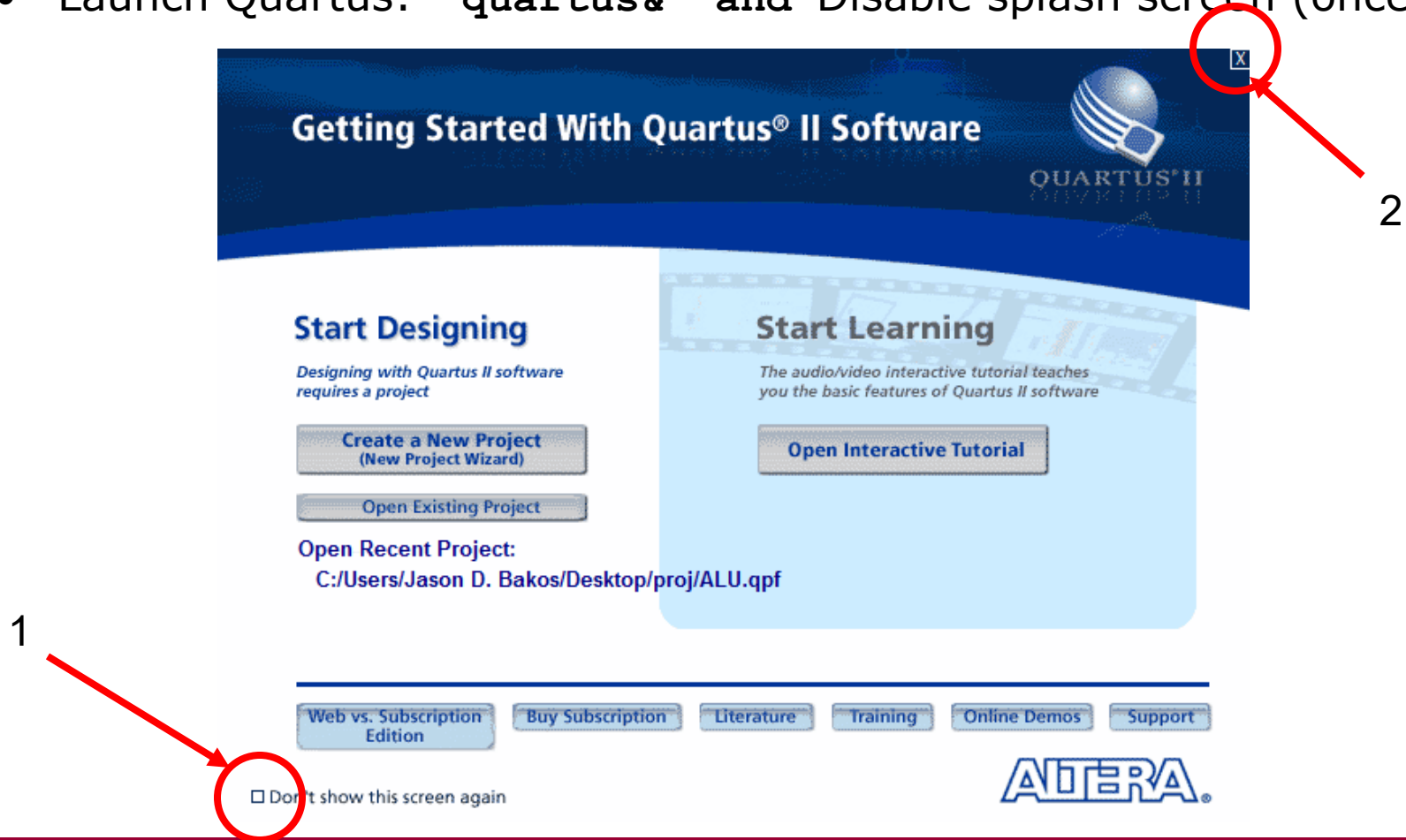
- Can also create a script for Modelsim:
- Example: test.do:

```
restart -f
force -freeze clk 1 0, 0 {50 ps} -r 100
force rst 1
force push 0
force pop 0
force data_in X"00000000"
run
force rst 0
force data_in X"00DECADE"
force push 1
run
force data_in X"00DEFACE"
run
```

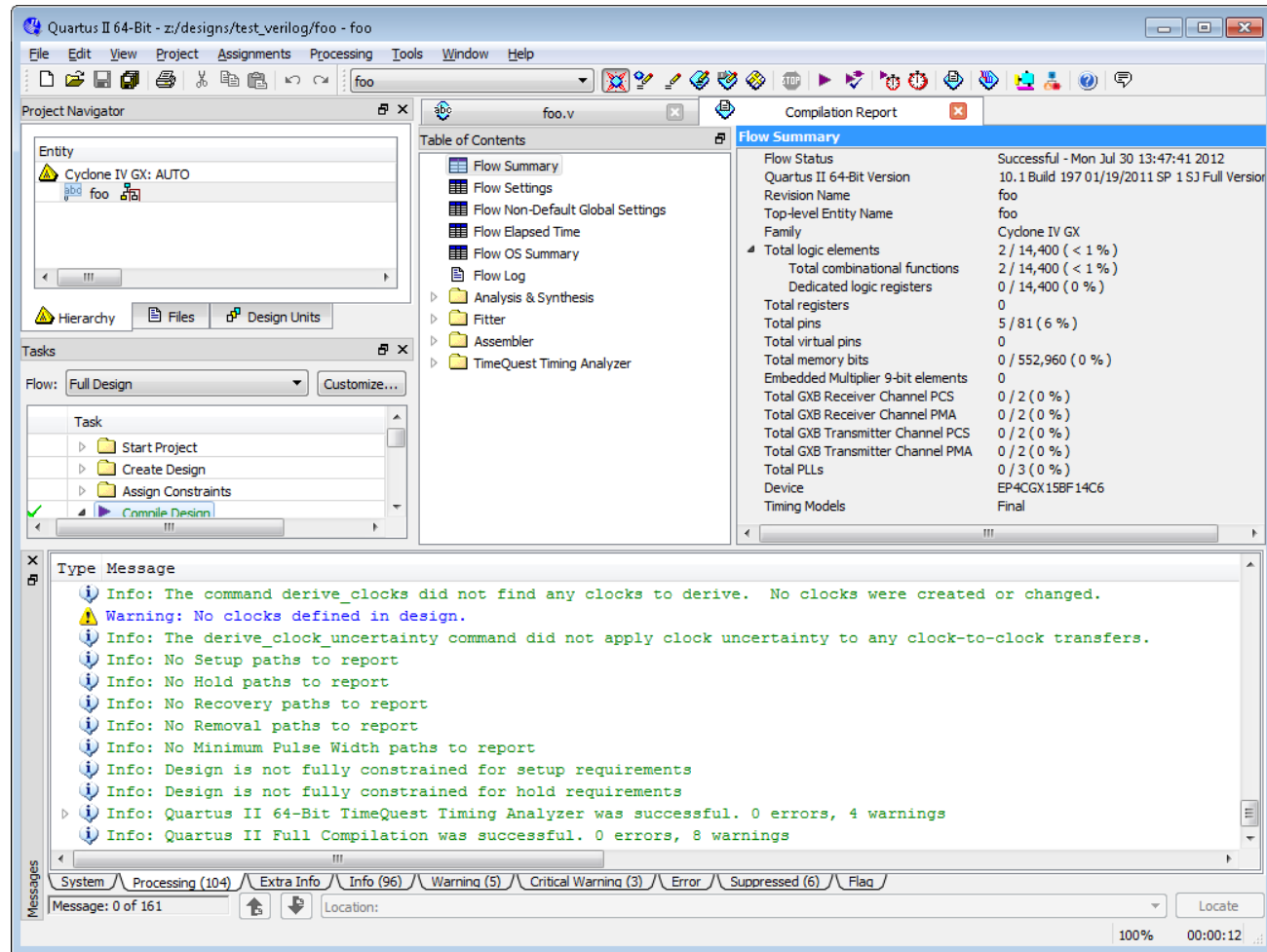
```
force data_in X"0000FADE"
run
force data_in X"00DECAFE"
run
force pop 1
force push 0
run
run
run
force pop 0
run
```

Quartus

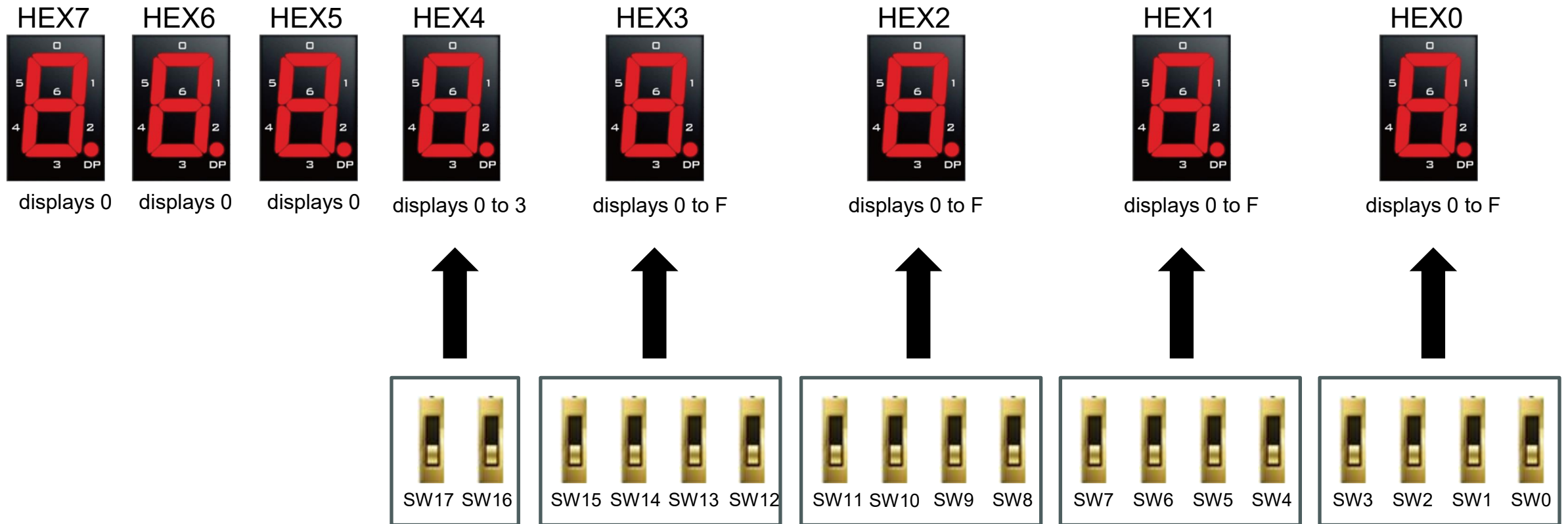
- Launch Quartus: "`quartus&`" and Disable splash screen (once)



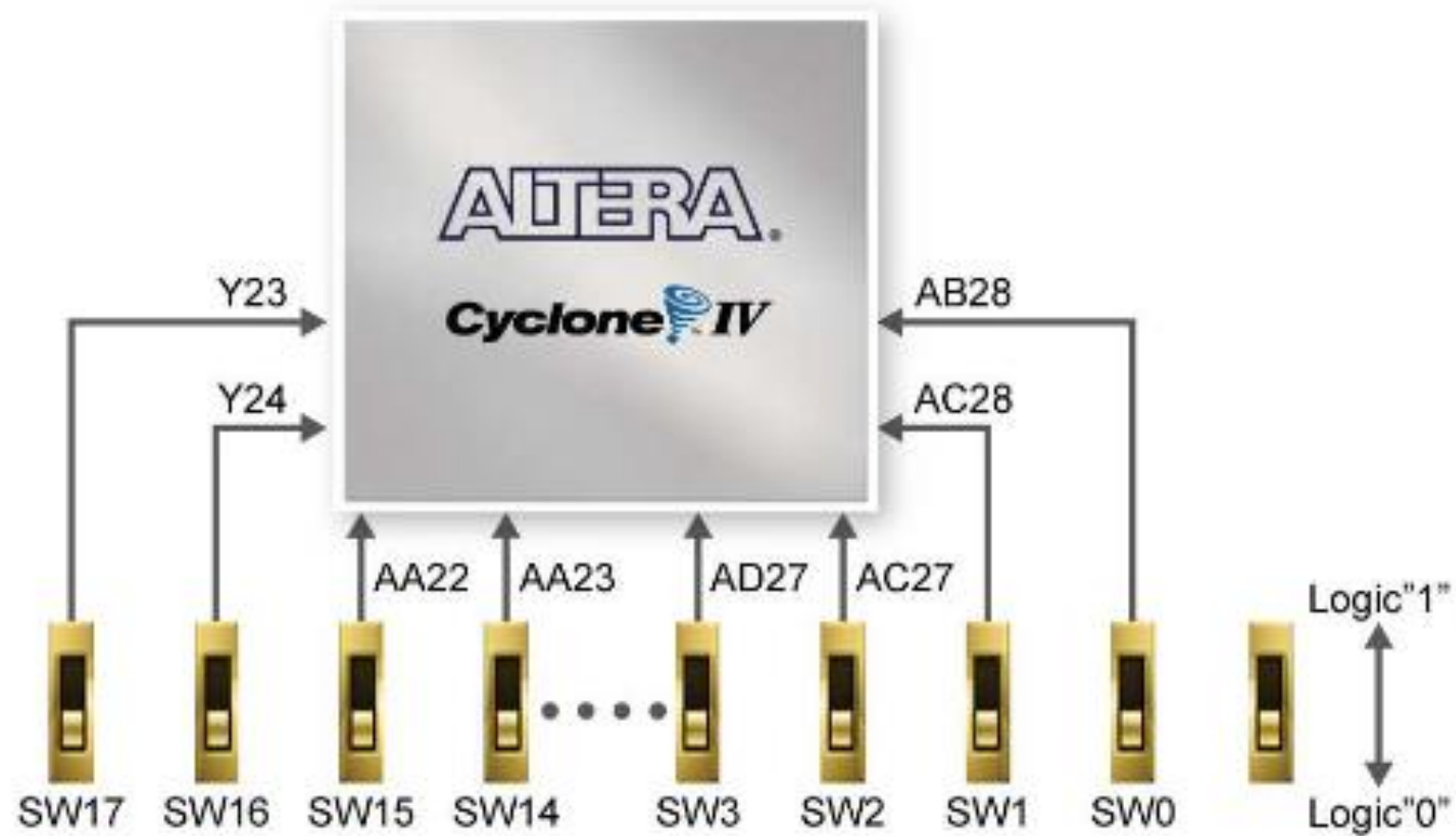
Quartus



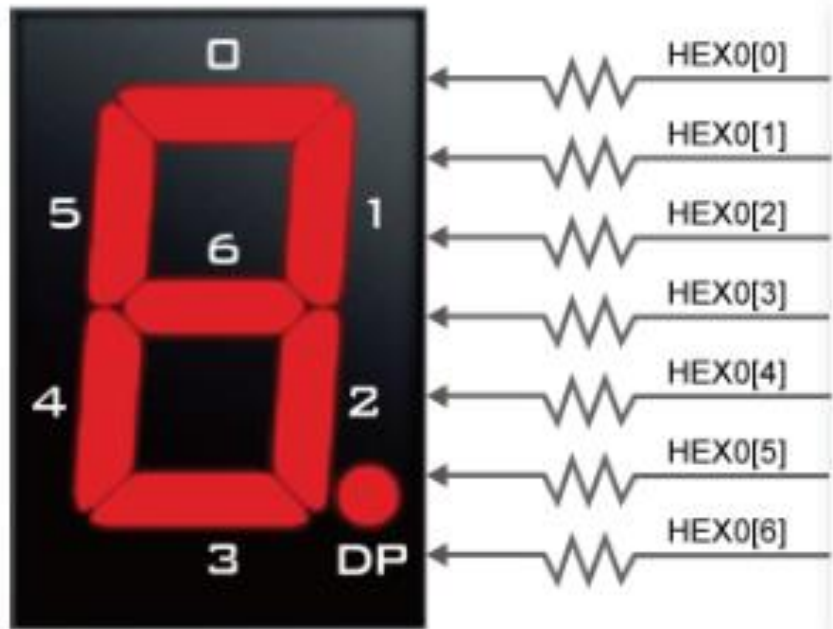
Lab 2



DE2-115 Board Switches



DE2-115 HEX Displays

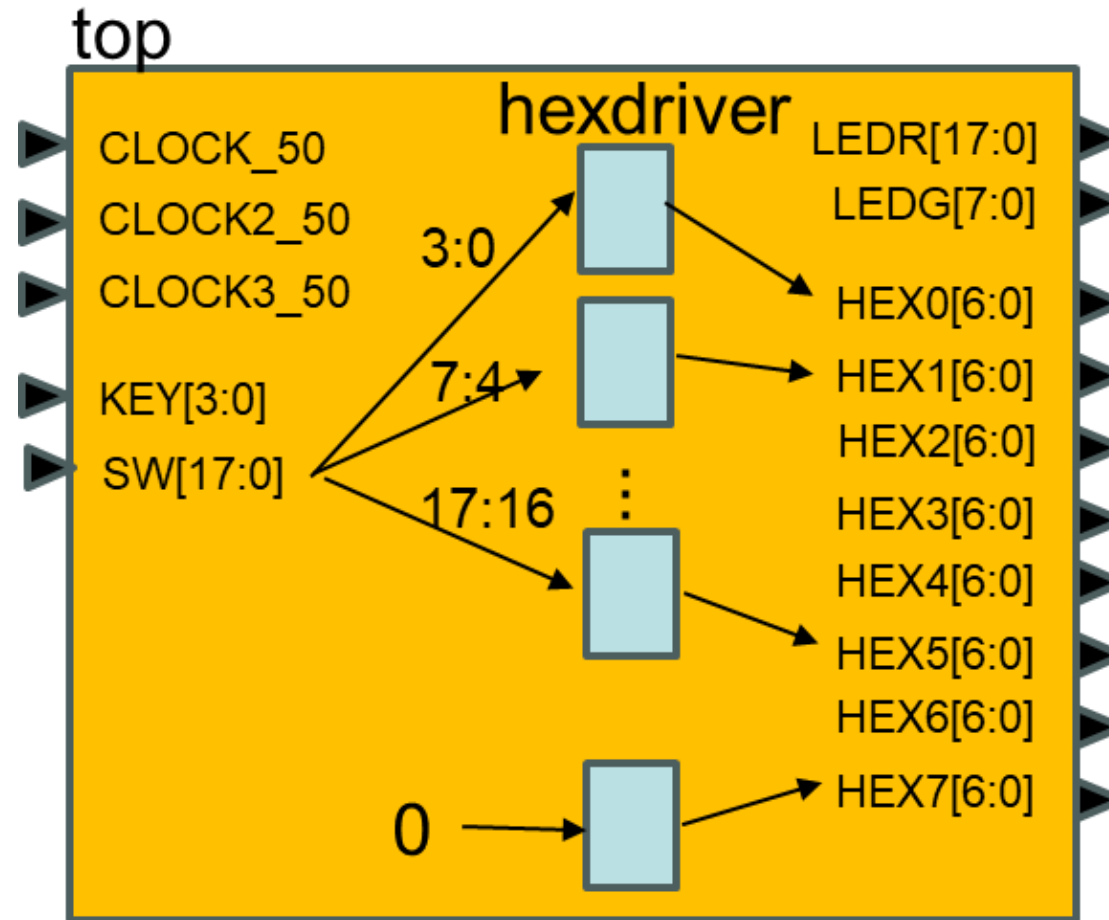


logic [6:0] HEX0,
logic [6:0] HEX1,
...
logic [6:0] HEX7,

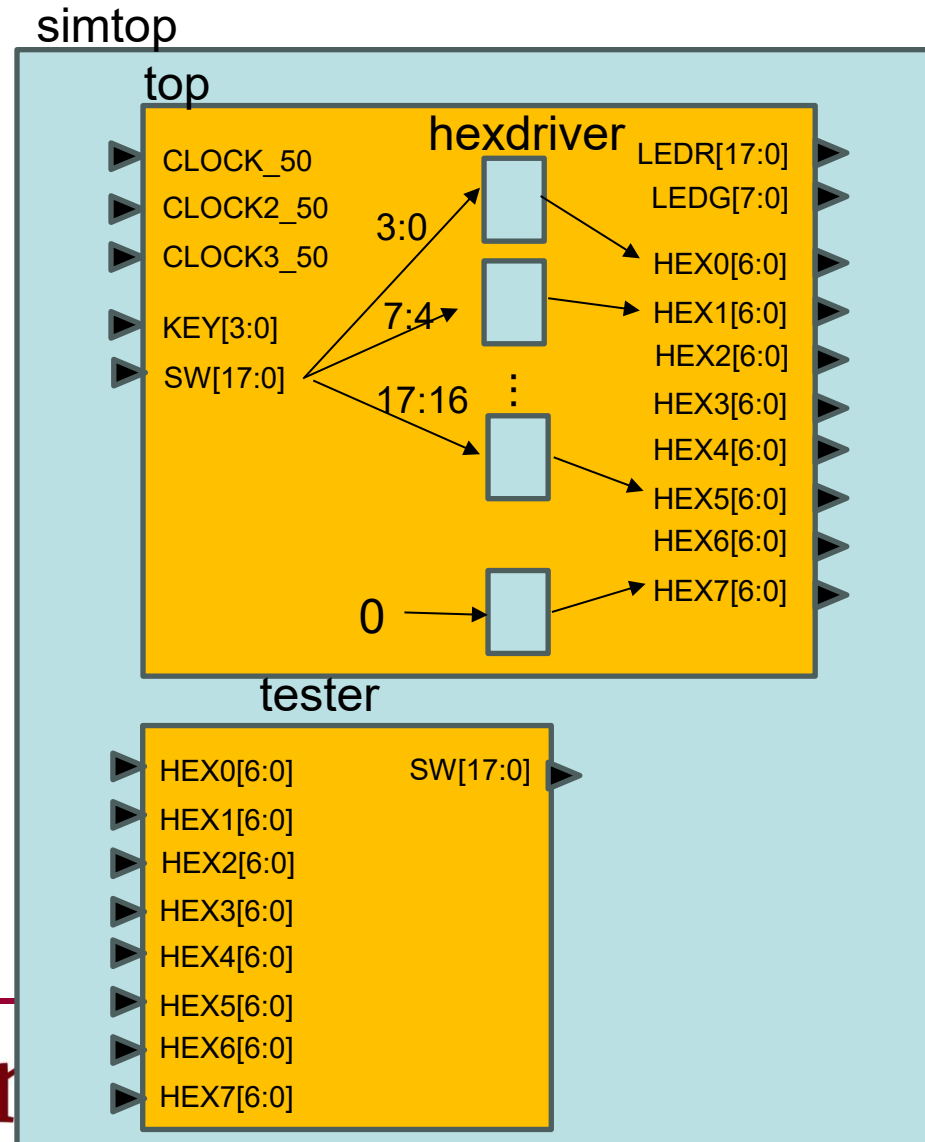
56 total bits

Signals are active-low (0 to turn on, 1 to turn off)

Top-Level Design on FPGA "top"

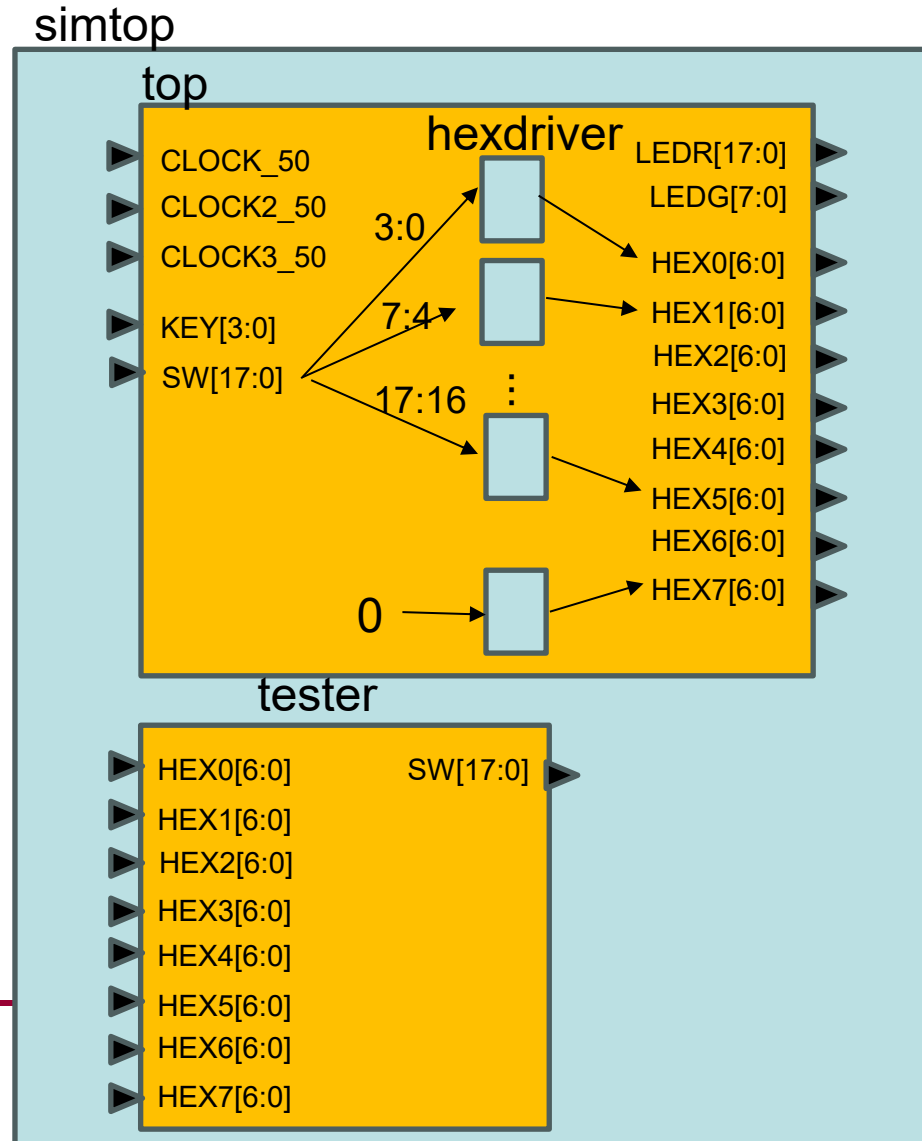


Top-Level Design for Simulation "simtop"



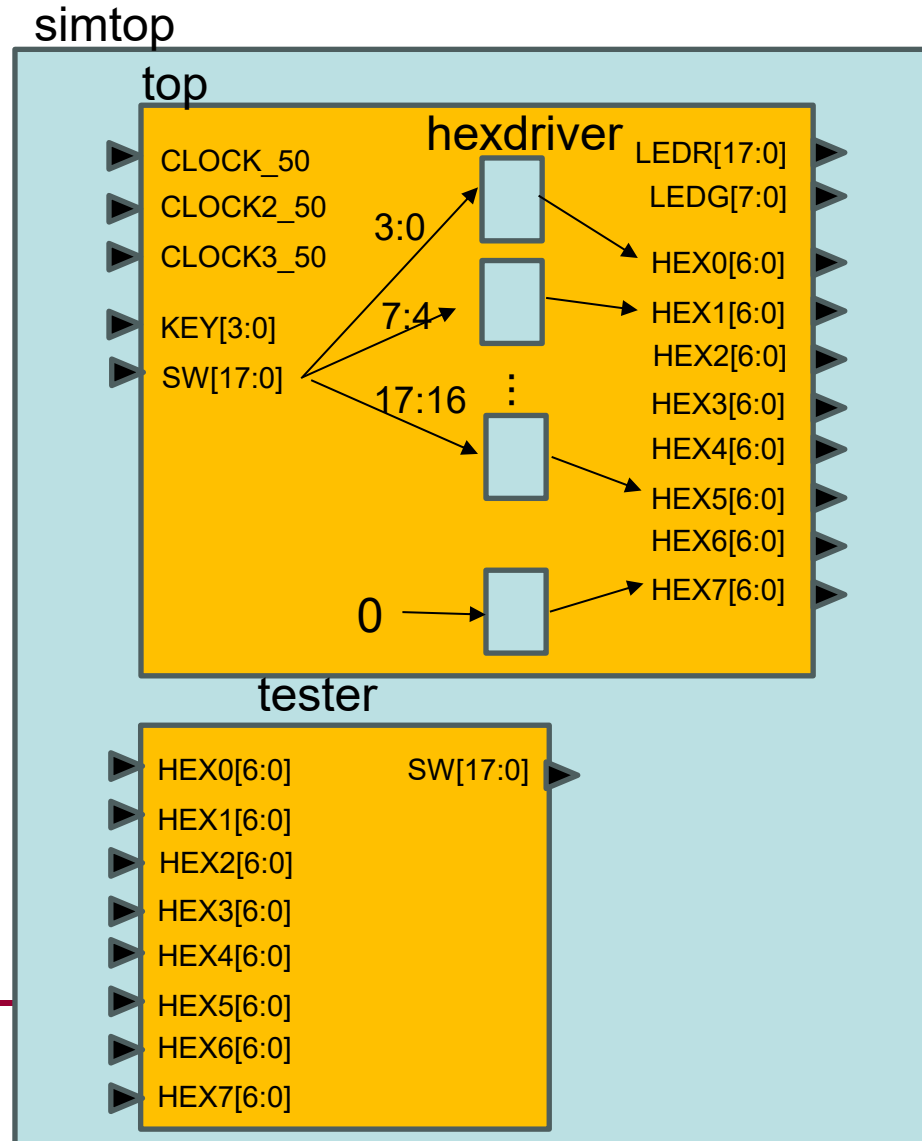
Lab 2 Skeleton

- Design files:
 - simtop.sv: top-level file for simulation
 - contains testbench
 - instances top module
 - top.sv: top-level file for synthesis
 - contains top-level pin out to board
 - also contains test code that lights up the LEDs in a shifting pattern
 - instance your hexdriver in this file
 - hexdriver.sv
 - Enter your decoder logic in this file



Lab 2 Skeleton

- Design files:
 - simtop.sv: top-level file for simulation
 - contains testbench
 - instances top module
 - top.sv: top-level file for synthesis
 - contains top-level pin out to board
 - also contains test code that lights up the LEDs in a shifting pattern
 - instance your hexdriver in this file
 - hexdriver.sv
 - Enter your decoder logic in this file



Lab 2 Skeleton

- To run testbench (no GUI):
 - `./csce611.sh testbench`
- To program FPGA:
 - `./csce611.sh program`
- To prepare for submission:
 - `./csce611.sh pack`

Editor Settings

- Editor settings
 - Go to Tools | Options | Text Editor | Autocomplete Text
 - Turn off!
- Set up FPGA:
 - EP4CE115F29C7
- Set up pins:
 - Assignments | Import Assignments
 - /usr/local/3rdparty/csce611/CPU_support_files/DE2_115_pin_assignments.qsf

Pin Mappings

Pin Planner - Z:/data/designs/lfsr/LSFR_testing - DE2_default

Top View - Wire Bond
Cyclone IVE - EP4CE115F29C7

Groups
Named: *

Node Name	Direction	Location
<new group>		

Report
Report not available

Tasks
Run Analysis and Elaboration
Early Pin Planning
Early Pin Planning...
Run I/O Assignment Analysis
Export Pin Assignments...

Filter: Pins: all

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Fitter Location	Current Strength
SW[9]	Input	PIN_AB25	5	B5_N1	2.5 V		PIN_AB25	8mA (default)
SW[10]	Input	PIN_AC24	5	B5_N2	2.5 V		PIN_AC24	8mA (default)
SW[11]	Input	PIN_AB24	5	B5_N2	2.5 V		PIN_AB24	8mA (default)
SW[12]	Input	PIN_AB23	5	B5_N2	2.5 V		PIN_AB23	8mA (default)
SW[13]	Input	PIN_AA24	5	B5_N2	2.5 V		PIN_AA24	8mA (default)
SW[14]	Input	PIN_AA23	5	B5_N2	2.5 V		PIN_AA23	8mA (default)
SW[15]	Input	PIN_AA22	5	B5_N2	2.5 V		PIN_AA22	8mA (default)
SW[16]	Input	PIN_Y24	5	B5_N2	2.5 V		PIN_Y24	8mA (default)
SW[17]	Input	PIN_Y23	5	B5_N2	2.5 V		PIN_Y23	8mA (default)
DRAM_ADDR[0]	Unknown	PIN_R6	2	B2_N0	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[1]	Unknown	PIN_V8	2	B2_N1	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[2]	Unknown	PIN_U8	2	B2_N1	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[3]	Unknown	PIN_P1	1	B1_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[4]	Unknown	PIN_V5	2	B2_N1	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[5]	Unknown	PIN_W8	2	B2_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[6]	Unknown	PIN_W7	2	B2_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[7]	Unknown	PIN_AA7	2	B2_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[8]	Unknown	PIN_Y5	2	B2_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[9]	Unknown	PIN_Y6	2	B2_N2	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[10]	Unknown	PIN_R5	2	B2_N0	3.3-V LVTTTL			8mA (default)
DRAM_ADDR[11]	Unknown	PIN_AA5	2	B2_N2	3.3-V LVTTTL			8mA (default)

0% 00:00:00

Recap

- Structural HDL
 - Instantiating modules inside of modules
 - Port maps
 - Signal renaming across the hierarchy
- Behavioral HDL
 - Continual assignment statements
- Miscellaneous
 - Making muxes and lookup tables with nested ternary statements

```
assign HEX_out = input_bits == 4'd0 ? 7'b1011100 :  
                    input_bits == 4'd1 ? 7'b1100000 :  
                    7'b1000000;
```
 - Numeric constants, e.g. `4'd8`
 - Bit slicing (e.g. `a[2:0]`), bit combining (e.g. `{a[0],a[1]}`), duplication (e.g. `{4{a[1]}}`)
 - Reduction operators (e.g. `y = &a`)

Always Statement

- Need a mechanism for creating:
 1. Combinational logic with if-statements, loops, and case statements
`always_comb`
 statements
 2. Memory: flip-flops, registers, state machines, RAMs
`always_ff @(sensitivity list, e.g. posedge clk)`
 statement;
 3. Verilog-style:
 `always @(sensivity list)`
 4. Non synthesizable behaviors, e.g. clocks that repeat forever
`always`
 statement;

Always Statement

- Always statements allow additional programming constructs not available with assign statement:
 - if statement
 - case statement
 - loops (probably won't use in this class)

```
logic out1;  
assign out1 = sel ? foo : bar;
```

— or —

```
logic out1;  
always_comb begin  
    if (sel) out1 = foo;  
    else out1 = bar;  
end
```

Always Statement

- Outside of an always statement, only allowed code:
 - Structure (module instantiations)
 - Continuous assignment "assign" statements
- Assigning a signal in an always block makes it the "driver", so don't drive it with any other always blocks, assigns, or modules
 - Can still be driven by an 'initial' block

Default Assignments

- For combinational logic, make sure all output signals are assigned on every control path
- Can use default signals to ensure this:

```
module default_assignment_test (input logic sel, foo, bar,  
    output logic out1);
```

```
    always_comb begin  
        out1 = bar;  
        if (sel==1) out1 = foo;  
    end
```

```
endmodule
```

Combinational Logic using `always`

```
// combinational logic using an always statement
module gates(input logic [3:0] a, b,
              output logic [3:0] y1, y2, y3, y4, y5);
    always_comb // need begin/end because there is
    begin      // more than one statement in always
        y1 = a & b; // AND
        y2 = a | b; // OR
        y3 = a ^ b; // XOR
        y4 = ~(a & b); // NAND
        y5 = ~(a | b); // NOR
    end
endmodule
```

- This hardware could be described with assign statements using fewer lines of code, so it's better to use assign statements in this case

Combinational Logic using case

```
module sevenseg(input logic [3:0] data,
                output logic [6:0] segments);

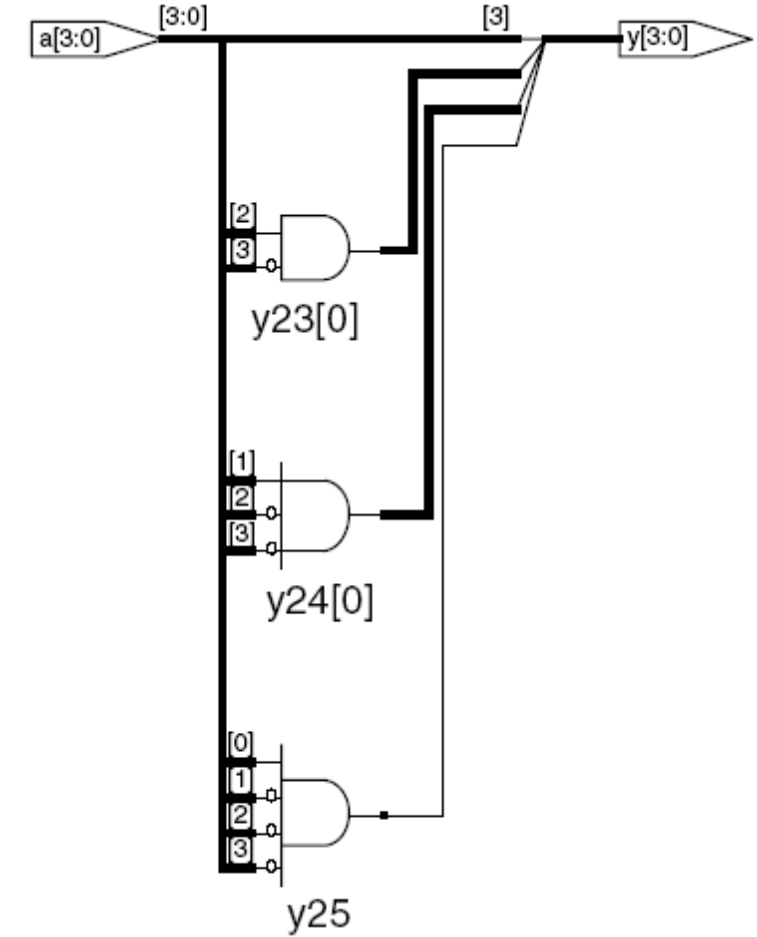
  always_comb
  case (data)
    //          abc_defg
    4'd0: segments = 7'b111_1110;
    4'd1: segments = 7'b011_0000;
    4'd2: segments = 7'b110_1101;
    4'd3: segments = 7'b111_1001;
    4'd4: segments = 7'b011_0011;
    4'd5: segments = 7'b101_1011;
    4'd6: segments = 7'b101_1111;
    4'd7: segments = 7'b111_0000;
    4'd8: segments = 7'b111_1111;
    4'd9: segments = 7'b111_1011;
    default: segments = 7'b000_0000; // required
  endcase
endmodule
```

Combinational Logic using **case**

- All possible input combinations must have a corresponding output assignment in the HDL
- Remember to use a default statement when necessary
- Use begin/end for multiple statements

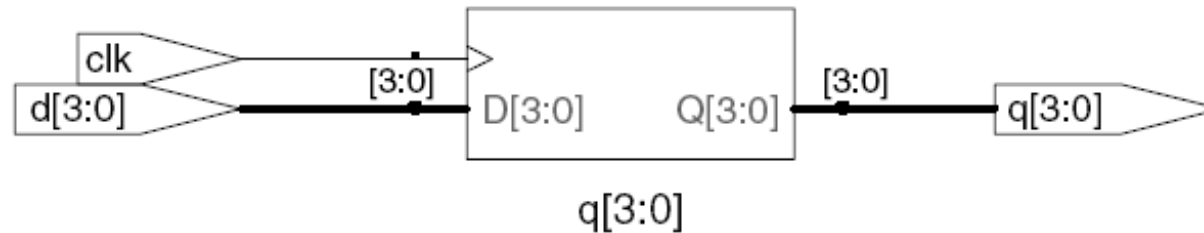
Combinational Logic using `casez`

```
module priority_casez(input logic [3:0] a,  
                     output logic [3:0] y);  
  
    always_comb  
        casez(a)  
            4'b1???: y = 4'b1000; // ? = don't care  
            4'b01??: y = 4'b0100;  
            4'b001?: y = 4'b0010;  
            4'b0001: y = 4'b0001;  
            default: y = 4'b0000;  
        endcase  
  
endmodule
```



Sequential Logic: D Flip-Flop

```
module flop(input          clk,  
            input          [3:0] d,  
            output logic [3:0] q);  
  
    always_ff @(posedge clk)  
        q <= d;                // pronounced "q gets d"  
  
endmodule
```

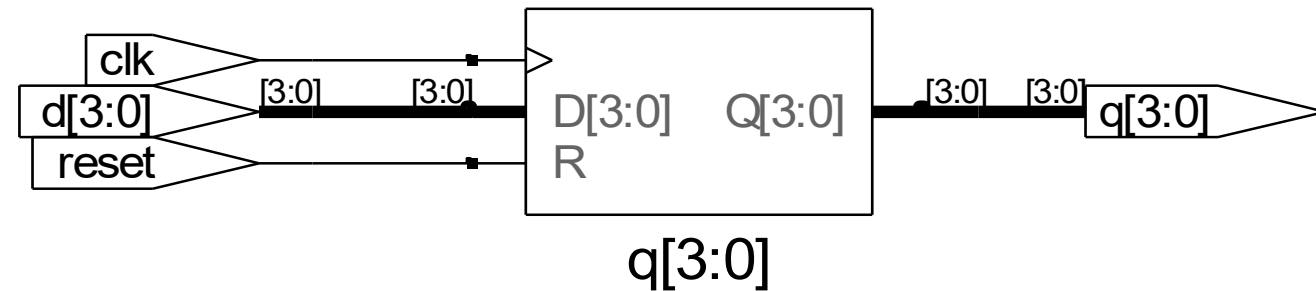


Resettable D Flip-Flop

```
module flopr(input      clk,
             input      reset,
             input [3:0] d,
             output logic [3:0] q);

    // synchronous reset
    always_ff @(posedge clk)
        if (reset) q <= 4'b0;
        else      q <= d;

endmodule
```

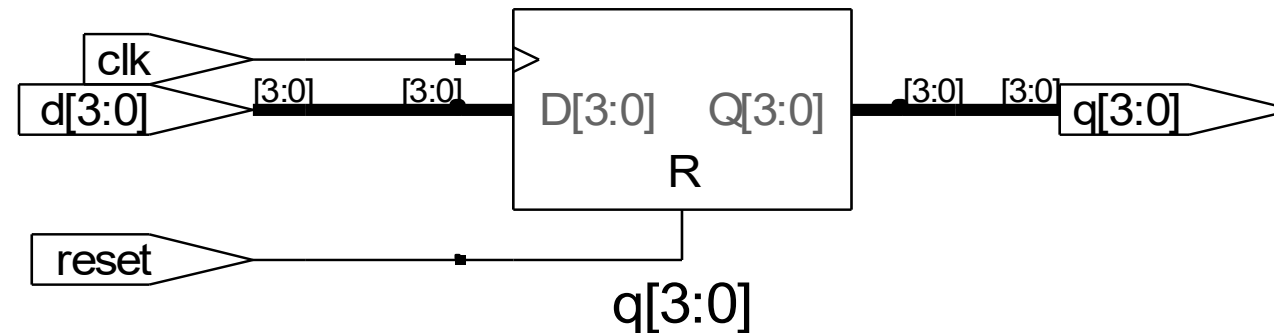


Resettable D Flip-Flop

```
module flopr(input      clk,
             input      reset,
             input      [3:0] d,
             output logic [3:0] q);

    // asynchronous reset
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else      q <= d;

endmodule
```

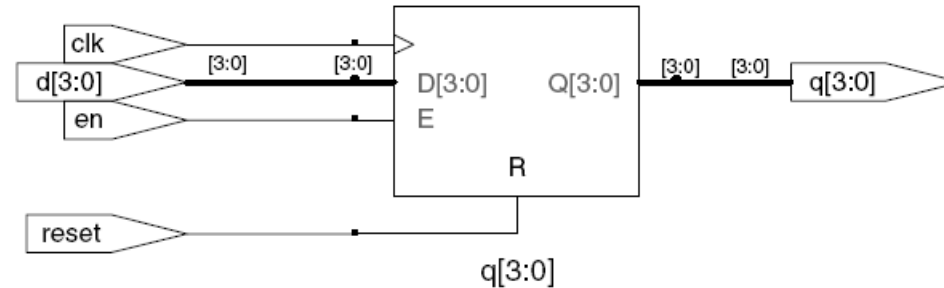


D Flip-Flop with Enable

```
module flopren(input      clk,
               input      reset,
               input      en,
               input [3:0] d,
               output logic [3:0] q);

    // asynchronous reset and enable
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else if (en) q <= d;

endmodule
```

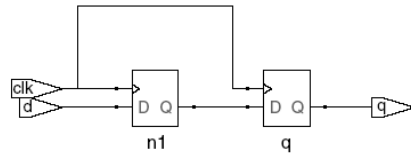


Blocking vs. Nonblocking Assignments

- `<=` is a “nonblocking assignment”
 - Occurs simultaneously with others (takes effect at end)
- `=` is a “blocking assignment”
 - Occurs in the order it appears in the file
 - Uses sequential semantics, use to build serialized paths

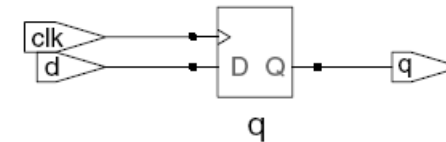
```
// Good synchronizer using
// nonblocking assignments
module syncgood(input      clk,
                input      d,
                output logic q);

  logic n1;
  always_ff @(posedge clk)
  begin
    n1 <= d; // nonblocking
    q  <= n1; // nonblocking
  end
endmodule
```



```
// Bad synchronizer using
// blocking assignments
module syncbad(input      clk,
               input      d,
               output logic q);

  logic n1;
  always_ff @(posedge clk)
  begin
    n1 = d; // blocking
    q  = n1; // blocking
  end
endmodule
```



Rules for Signal Assignment

- Use `always_ff @(posedge clk)` and nonblocking assignments (`<=`) to model synchronous sequential logic

```
always_ff @ (posedge clk)
    q <= d; // nonblocking
```

- Use continuous assignments (`assign ...`) to model simple combinational logic.

```
assign y = a & b;
```

- Use `always_comb` and blocking assignments (`=`) to model more complicated combinational logic where the `always` statement is helpful.
- Do not make assignments to the same signal in more than one `always` statement or continuous assignment statement

Testbenches

- HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)
- Generally not synthesizable
- Types of testbenches:
 - Simple testbench
 - Self-checking testbench
 - Self-checking testbench with testvectors

Example

```
module sillyfunction(input  a, b, c,  
                      output y);  
    assign y = ~b & ~c | a & ~b;  
endmodule
```

Simple Testbench

```
module testbench1();  
    logic a, b, c;  
    logic y;  
    // instantiate device under test  
    sillyfunction dut(.a(a), .b(b), .c(c), .y(y));  
    // apply inputs one at a time  
    initial begin  
        a = 1'b0; b = 1'b0; c = 1'b0; #10;  
        c = 1'b1; #10;  
        b = 1'b1; c = 1'b0; #10;  
        c = 1'b1; #10;  
        a = 1'b1; b = 1'b0; c = 1'b0; #10;  
        c = 1'b1; #10;  
        b = 1'b1; c = 1'b0; #10;  
        c = 1'b1; #10;  
    end  
endmodule
```

Self-checking Testbench

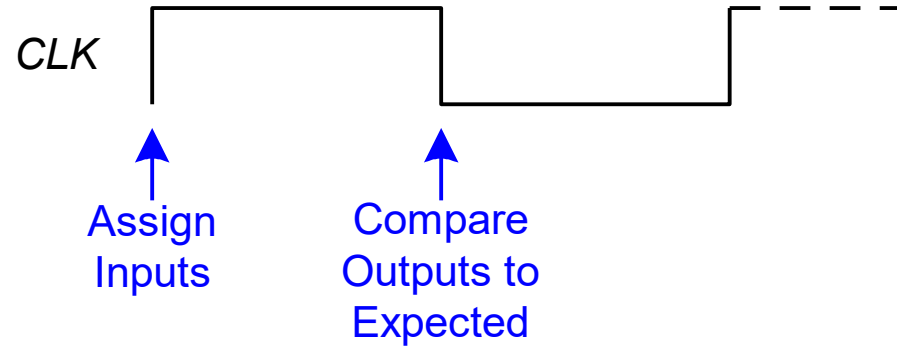
```
module testbench2();
    reg  a, b, c;
    wire y;
    // instantiate device under test
    sillyfunction dut(a, b, c, y);
    // apply inputs one at a time
    // checking results
    initial begin
        a = 1'b0; b = 1'b0; c = 1'b0; #10;
        if (y !== 1'b1) $display("000 failed.");
        c = 1'b1; #10;
        if (y !== 1'b0) $display("001 failed.");
        b = 1'b1; c = 1'b0; #10;
        if (y !== 1'b0) $display("010 failed.");
        c = 1'b1; #10;
        if (y !== 0) $display("011 failed.");
        a = 1'b1; b = 1'b0; c = 1'b0; #10;
        if (y !== 1'b1) $display("100 failed.");
        c = 1'b1; #10;
        if (y !== 1) $display("101 failed.");
        b = 1'b1; c = 1'b0; #10;
        if (y !== 1'b0) $display("110 failed.");
        c = 1'b1; #10;
        if (y !== 1'b0) $display("111 failed.");
    end
endmodule
```

Testbench with Testvectors

- Write testvector file: inputs and expected outputs
- Testbench:
 1. Generate clock for assigning inputs, reading outputs
 2. Read testvectors file into array
 3. Assign inputs, expected outputs
 4. Compare outputs to expected outputs and report errors

Testbench with Testvectors

- Testbench clock is used to assign inputs (on the rising edge) and compare outputs with expected outputs (on the falling edge).



- The testbench clock may also be used as the clock source for synchronous sequential circuits

Testvectors File

File: `example.tv` - contains vectors of `abc_yexpected`

`000_1`

`001_0`

`010_0`

`011_0`

`100_1`

`101_1`

`110_0`

`111_0`

Testbench: 1. Generate Clock

```
module testbench3();  
    logic          clk, reset;  
    logic          a, b, c, yexpected;  
    logic          y;  
    logic [31:0] vectornum, errors;    // bookkeeping variables  
    logic [3:0] testvectors[10000:0]; // array of testvectors  
  
    // instantiate device under test  
    sillyfunction dut(.a(a), .b(b), .c(c), .y(y));  
  
    // generate clock  
    always // no sensitivity list, so it always executes  
    begin  
        clk = 1'b1; #5; clk = 1'b0; #5;  
    end
```

2. Read Testvectors into Array

```
// at start of test, load vectors
// and pulse reset

initial
begin
    $readmemb("example.tv", testvectors);
    // note: use $readmemh() for ALU testbench
    vectornum = 32'b0; errors = 32'b0;
    reset = 1'b1; #27; reset = 1'b0;
end

// Note: $readmemh reads testvector files written in
// hexadecimal (as opposed to $readmemb in binary)
// Can insert Verilog comments into file
// Can also add addresses (use @ with hex address)
```

3. Assign Inputs and Expected Outputs

```
// apply test vectors on rising edge of clk
always @(posedge clk)
    begin
        {a, b, c, yexpected} = testvectors[vectornum];
    end
```

4. Compare Outputs with Expected Outputs

```
// check results on falling edge of clk
always @(negedge clk)
    if (~reset) begin // skip during reset
        if (y !== yexpected) begin
            $display("Error: inputs = %b", {a, b, c});
            $display("  outputs = %b (%b expected)", y, yexpected);
            errors = errors + 32'b1;
        end
    end

// Note: to print in hexadecimal, use %h. For example,
//      $display("Error: inputs = %h", {a, b, c});
```

4. Compare Outputs with Expected Outputs

```
// increment array index and read next testvector
    vectornum = vectornum + 32'b1;
    if (testvectors[vectornum] === 4'bx) begin
        $display("%d tests completed with %d errors",
            vectornum, errors);
    $finish;
    end
end
endmodule
```

```
// Note: === and !== can compare values that are
// x or z.
```